RC 2025 智慧果园

RC 2025 智慧果园

- Part 0 基础操作知识
  - SubPart 1 底盘硬件相关
  - SubPart 2 上位机相关
    - Step 1 下载 & 安装
    - Step 2 烧录
    - Step 3 设备上电与联网
    - Step 4 远程连接
    - Step 5 基础操作
  - SubPart 3 代码架构简介
  - SubPart 4 机器人坐标系
- Part 1 硬件校准测试
  - Step 1 校准机械臂
  - Step 2 校准底盘
  - Step 3 相机测试/校准
- Part 2 基础功能测试
  - SubPart 1 二维码定位
  - SubPart 2 目标检测
    - Step 1 制作数据集
      - SubStep 1 安装 conda
      - SubStep 2 安装 labelimg
      - SubStep 3 **使用 labelimg**
    - Step 2 训练
    - Step 3 模型转换
      - SubStep 1 pt -> onnx
      - SubStep 2 onnx -> rknn
    - Step 4 部署
  - SubPart 3 大模型
- Part 3 综合功能测试
  - SubPart 1 放置区测序
  - SubPart 2 二维码定位导航
  - SubPart 3 水果采摘
    - Step 0 空间定位原理
    - Step 1 抓取测试
- Part 4 比赛任务全程联调

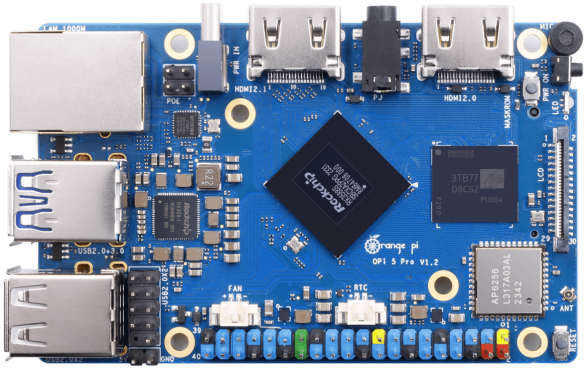
Part 0 基础操作知识

SubPart 1 底盘硬件相关

车体侧面有电压显示，调试时需关注电压，电压尽量**避免低于 11.1V**，及时充电

充电时为保证安全，请在有人员在场的情况下充电，虽然有过充保护，但是也尽量**避免过夜充电**

SubPart 2 上位机相关



http://www.orangepi.cn/html/hardWare/computerAndMicrocontrollers/details/Orange-Pi-5-Pro.html

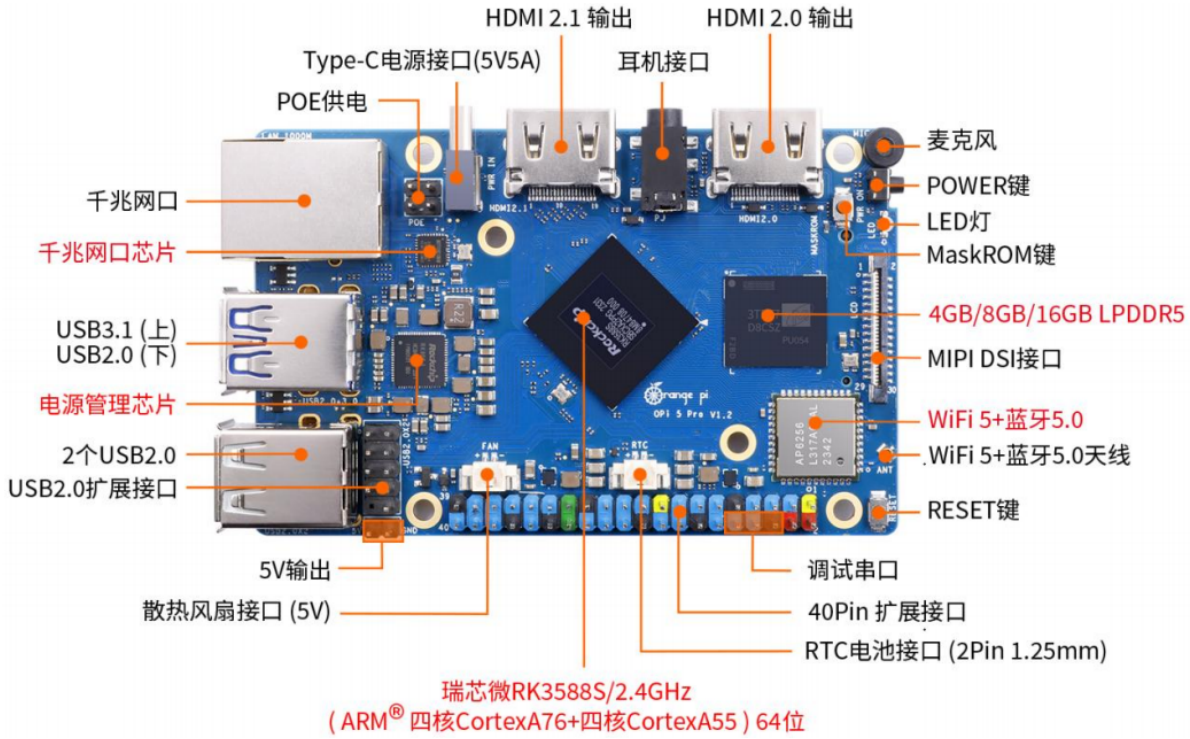
本产品配套 Orange Pi 5 Pro 8GB 版本作为上位机，使用 DC 5V/5A 的 Type-C 电源供电

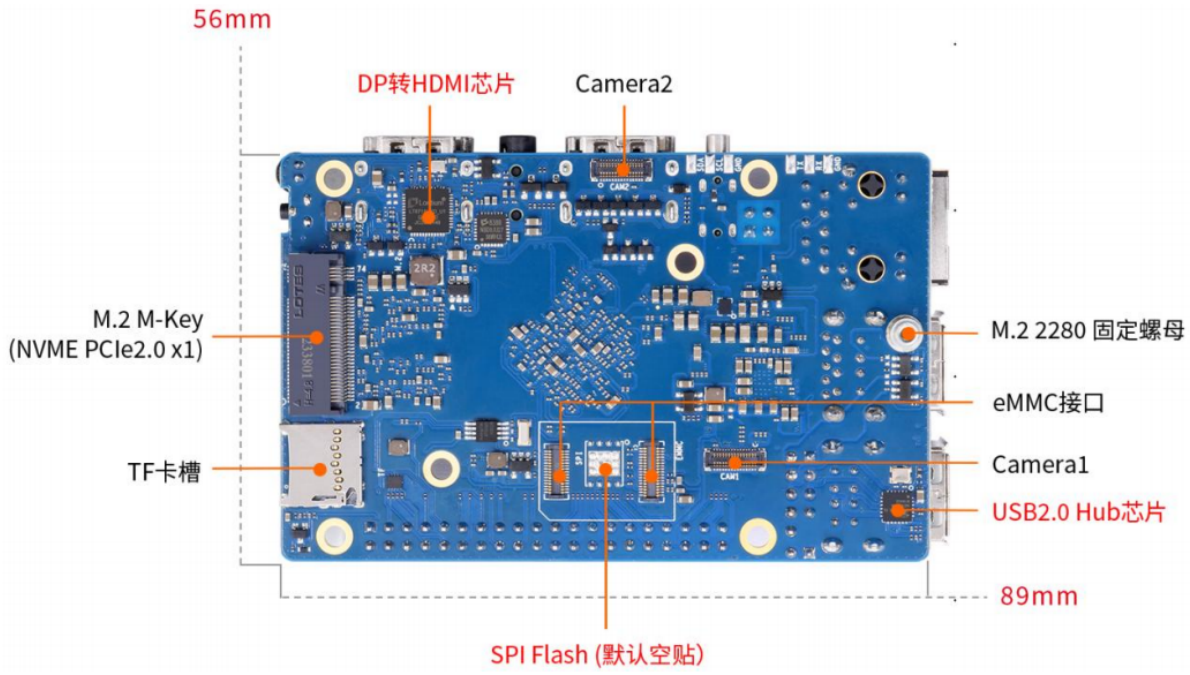
Orange Pi 5 Pro 采用了瑞芯微 RK3588S 新一代八核 64 位 ARM 处理器，具体为四核 A76 和四核 A55，采用的三星 8nm LP 制程工艺，大核主频最高可达 2.4GHz，集成 ARM Mali-G610 MP4 GPU，内嵌高性能 3D 和 2D 图像加速模块，内置高达 6Tops 算力的 AI 加速器 NPU，拥有 4GB/8GB/16GB（LPDDR5）内存，具有高达 8K 显示处理能力。

Orange Pi 5 Pro 引出了相当丰富的接口，包括 HDMI 输出、Wi-Fi5、M.2 PCIe2.0x1、千兆网口、支持 PoE+（需要 PoE+ HAT）、USB2.0、USB3.1 接口和 40pin 扩展排针等。可广泛适用于高端平板、边缘计算、人工智能、云计算、AR/VR、智能安防、智能家居等领域，覆盖 AIoT 各个行业。

Orange Pi 5 Pro 支持 Orange Pi 官方研发的操作系统 Orange Pi OS，同时，支持 Android 12.1、Debian11、Debian12、Ubuntu20.04 和 Ubuntu22.04 等操作系统。

硬件特性介绍	
CPU	• Rockchip RK3588S (8nm LP 制程） • 8 核 64 位处理器 • 4 核 Cortex-A76 和 4 核 Cortex-A55 大小核架构 • 大核主频最高 2.4GHz，小核主频最高 1.8GHz
GPU	• 集成 ARM Mali-G610 • OpenGL ES1.1/2.0/3.2、OpenCL 2.2 和 Vulkan 1.2
NPU	• 内置高达 6 Tops 算力的 AI 加速器 NPU • 支持 INT4/INT8/INT16 混合运算
视频输出	• HDMI 2.1，最大支持 8K @60Hz • HDMI 2.0，最大支持 4K@60Hz • 1* MIPI D-PHY TX 4Lane
内存	4GB/8GB/16GB（LPDDR5）
摄像头	• 1 * MIPI CSI 4Lane • 1 * MIPI D-PHY RX 4Lane
PMU	RK806-1
板载存储	• eMMC 插座，可外接 eMMC 模块 • 16MB QSPI Nor FLASH（默认空贴，与 eMMC 复用） • MicroSD (TF) Card 插槽 • PCIe2.0x1 M.2 M-KEY (SSD) 插槽
以太网	10/100/1000Mbps 以太网 ，支持 PoE+（需要 PoE+ HAT）
音频	• 3.5mm 耳机孔音频输入/输出 • 板载 MIC 输入 • 2 * HDMI 输出
PCIe M.2 M-KEY	• 支持 SSD
USB 接口	1 * USB3.0 支持 Device 或 HOST 模式 3 * USB2.0 HOST 2 * USB2.0 HOST（通过 9pin 排针引出）
40pin 扩展排针	用于扩展 UART、PWM、I2C、SPI、CAN 和 GPIO 接口
调试串口	包含在 40PIN 扩展口内
LED 灯	RGB LED 三色指示灯
按键	1 * MaskROM 键，1 * RESET 键，1 * 开关机键
供电	Type-C 接口供电 5V/5A；
支持的操作系统	Orange Pi OS（Droid）、Orange Pi OS（Arch）、Android12.1、Android13、Debian11、Debian12、Ubuntu20.04、Ubuntu22.04 等操作系统
外观规格介绍	
产品尺寸	89mm*56mm
重量	58g
range Pi™ 是深圳市迅龙软件有限公司的注册商标	





Step 1 下载 & 安装

本司提供修改过适配本产品的镜像



该镜像的系统中，计算机名：orangepi　密码：orangepi

下载该镜像

同时下载镜像烧录工具 balenaEtcher (https://etcher.balena.io/#download-etcher)

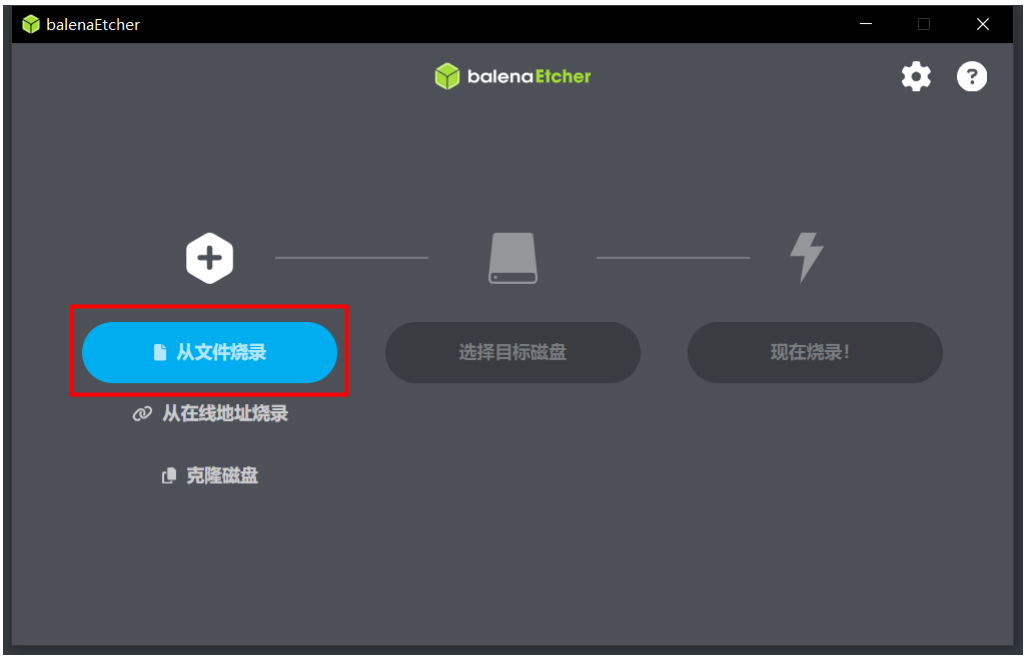


双击安装包安装 balenaEtcher

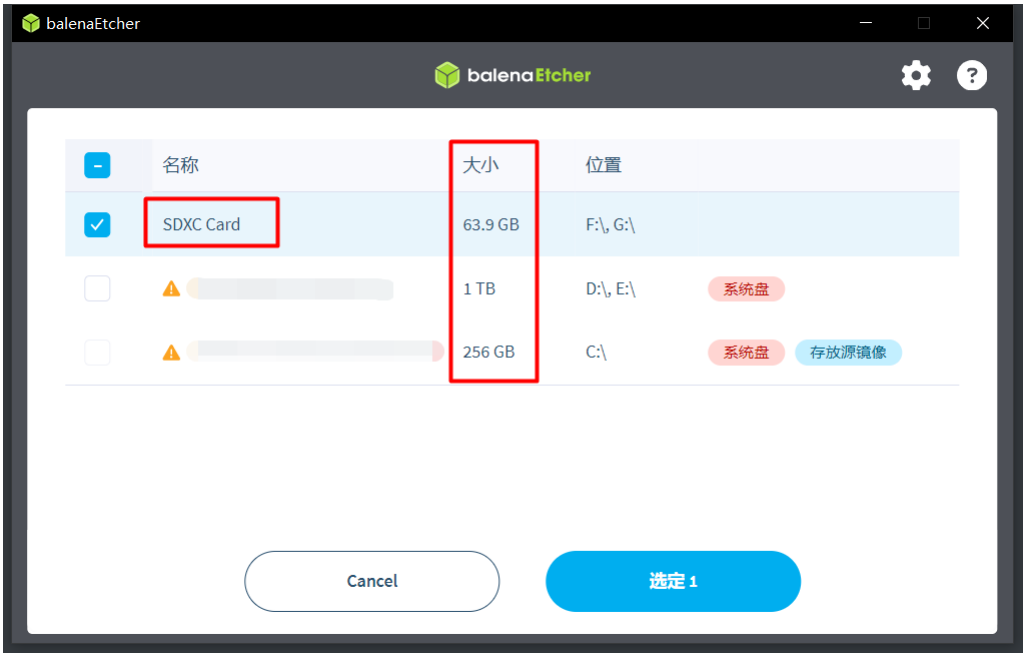
Step 2 烧录

将 SD 卡通过 USB 读卡器插入电脑

打开软件，点击 “从文件烧录”，选择

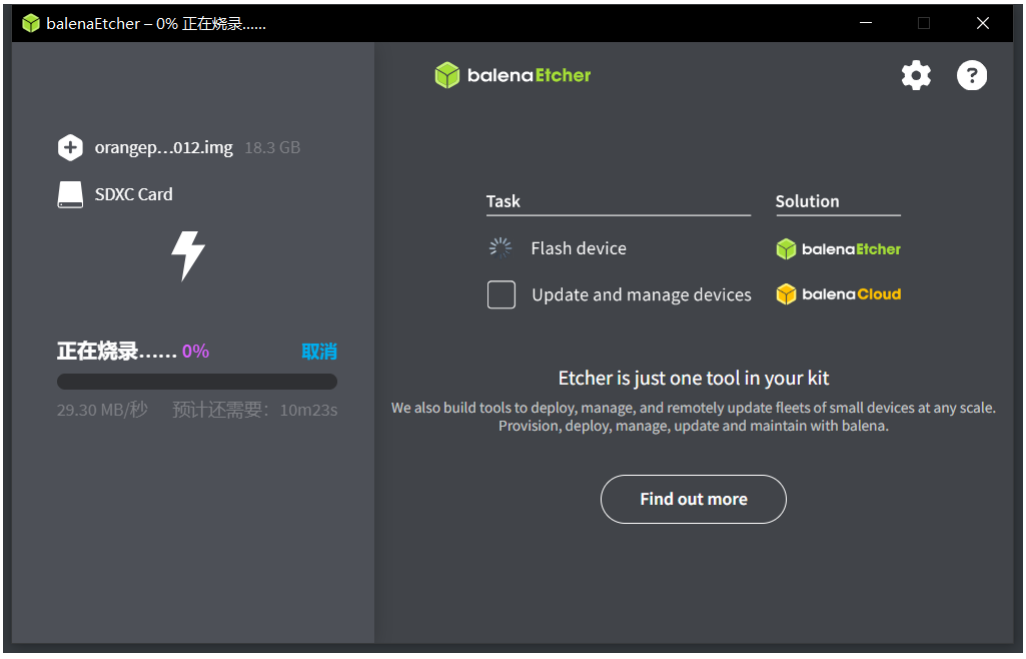


选择完成后，再点击 “选择目标磁盘”

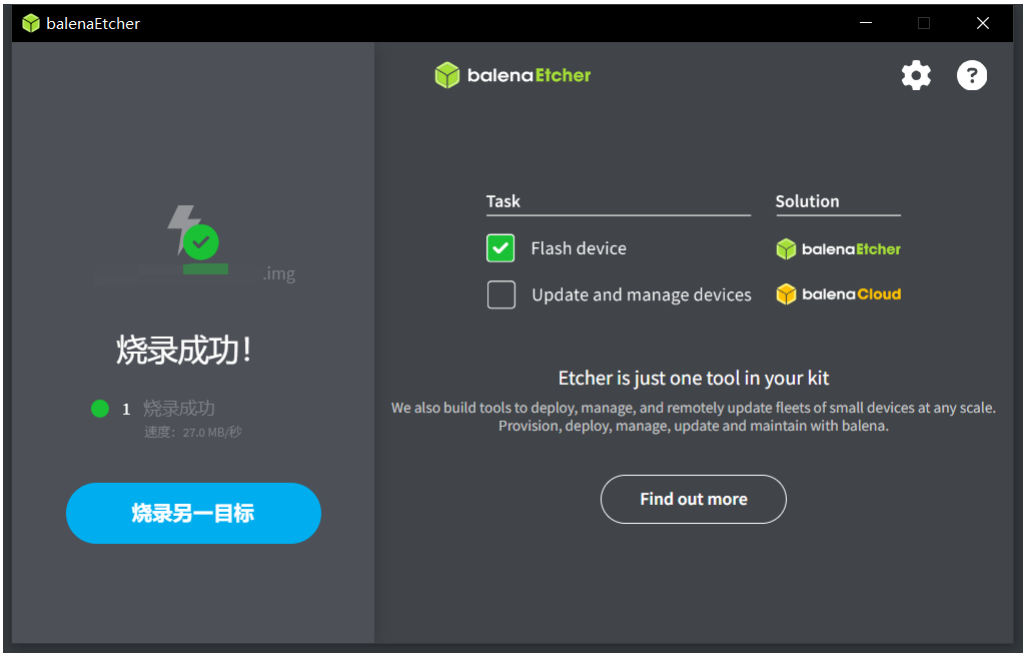


注意：会检测出多个磁盘，根据磁盘容量大小选择对应的 SD 卡，切勿选择电脑自己的磁盘，以免擦除电脑数据

选定后，点击 “现在烧录！”，会弹出 Windows 命令处理程序的提示，选择 是



耐心等待烧录与验证步骤完成



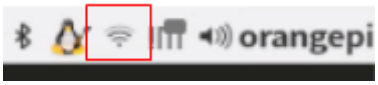
拔下读卡器，取出 SD 卡，至此镜像烧录完成

Step 3 设备上电与联网

确保烧录好镜像的 SD 卡插入卡槽（注意，卡槽不带按压锁扣，需确保插入到底）

上电后，Orange Pi 5 Pro 上位机的指示灯会开始闪烁（蓝灯双闪），此时需先使用 HDMI 线连接至显示屏（推荐使用独立的电脑显示屏），可看到系统画面（如指示灯已闪烁，但屏幕无显示，可以尝试更换 Orange Pi 5 Pro 上的 HDMI 插口）

接入键盘鼠标，点击右上角 WIFI 按钮，搜索网络，接入网络



然后再次点击 WIFI 按钮，下拉框中点击网络连接信息



记住被远程设备的 IP 地址



记录好 IP 地址后即可拔掉连接显示屏的 HDMI 线，换 HDMI 显卡欺骗器插在其中一个 HDMI 口上

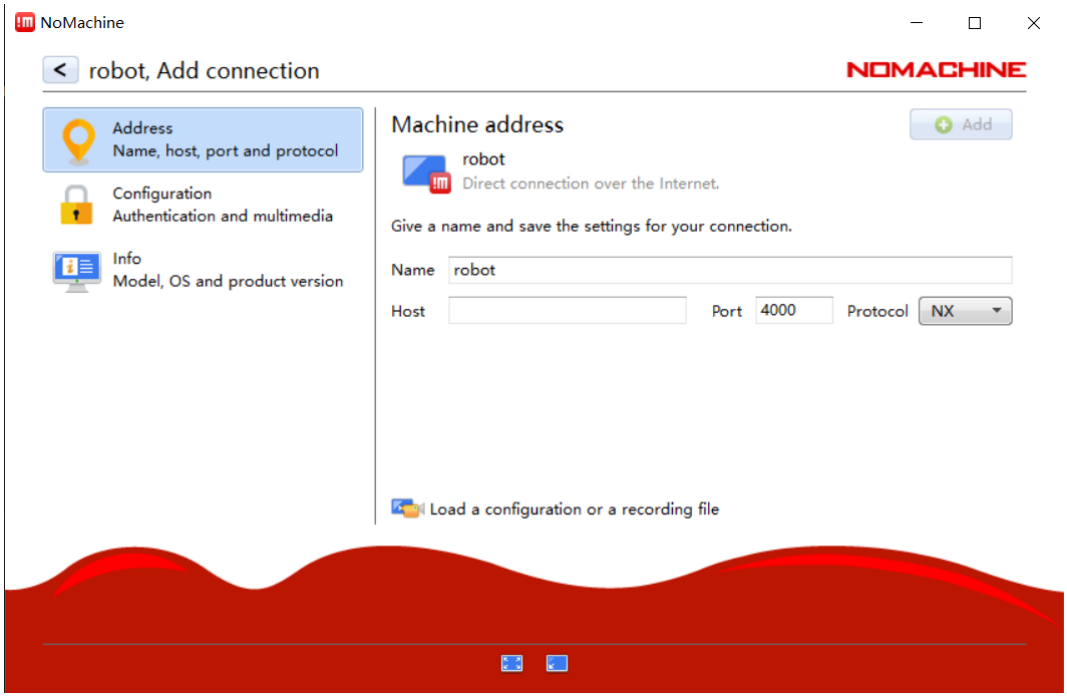
Step 4 远程连接

在 PC 上安装 NoMachine

注意：NoMachine 安装时，安装路径中不得出现中文

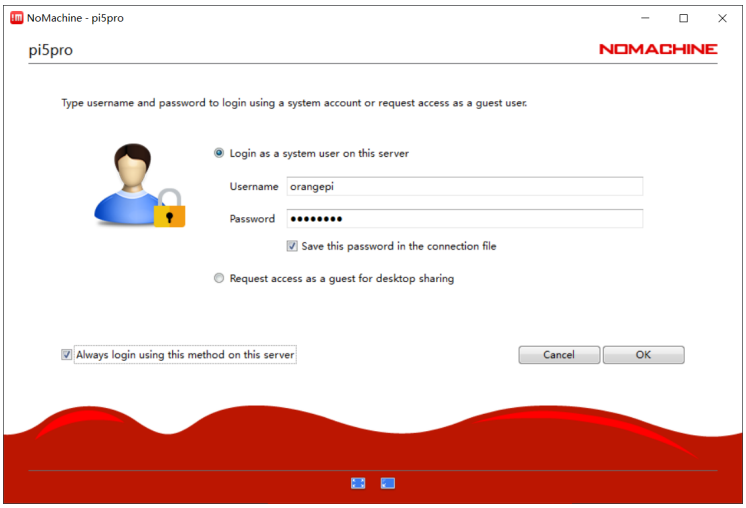


PC 端打开 NoMachine，点击主页面左上角的 Add 按钮新建连接，Name 可随意输入，Host 须输入上方记录的 IP 地址，填写完成后点击当前页面右上角 Add 按钮



在主页面中右击新建的连接（主页面中会存在两种连接图标，红色的为上述步骤自建的远程连接，黄色的是软件自动检索当前网络发现的设备），选择 Edit connection 配置自建连接的目标系统用户信息

针对本司提供的 OrangePi 5 pro 镜像，Username 栏须输入 orangepi，Password 为 orangepi



配置好目标系统用户信息后，可勾选下方的记住密码和总是以此方式登录，点击 OK，进行远程控制（第一次配置好后，若 IP 地址不变，后期可直接双击创建的远程连接）

远程连接成功后，即可在自己电脑上操作上位机系统

注意：经测试，OrangePi 5 Pro 必须有显示输出设备（显卡欺骗器也可）接在 HDMI 口上才能使用本赛项的一些功能，如远程连接后发现画面分辨率很小，或者运行中 RVIZ 无法打开，可以考虑更换 HDMI 插口再尝试

Step 5 基础操作



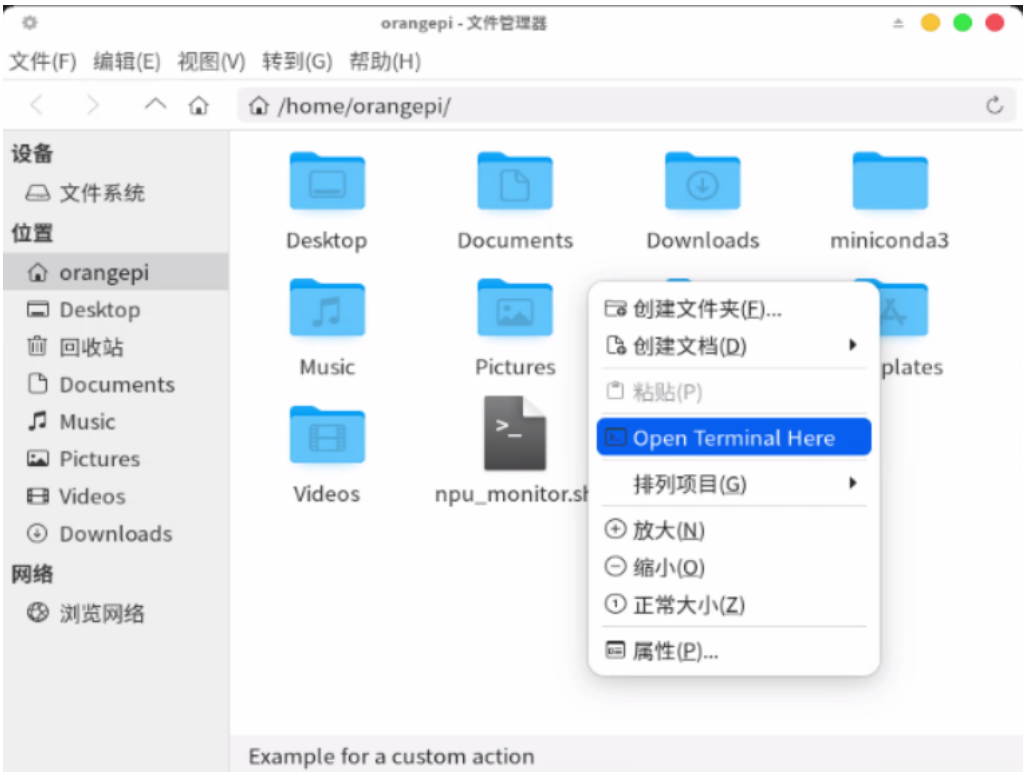
点击上图左上角所有应用程序可打开菜单选择应用

下方中间 Dock 栏预添加了常用应用，包括 Chrominum / 文件夹 / 终端 / 文本编辑器 / VsCode /设置 / 系统监视器 / CPU状态

桌面上有一个调节分辨率的脚本，用于在未插入 HDMI 欺骗器的时候正常显示（但打开 RVIZ / Gazebo 等时，需接屏幕或 HDMI 欺骗器，否则还是会报错）

打开终端

在任意界面内点击右键可以看到在终端打开选项



或者在下方的 Dock 栏中点击终端图标

如果需要打开多个终端，一方面可以重新右键打开，或者在 Dock 中用鼠标中键点击终端图标，或在当前打开的终端中按 Ctrl + Shift + t 打开新终端

在终端中，复制的快捷键为 Ctrl + Shift + c，粘贴的快捷键为 Ctrl + Shift + v

扩容

为缩减制作的镜像的大小，烧录后系统盘中还有大量未分配空间，需要手动分配

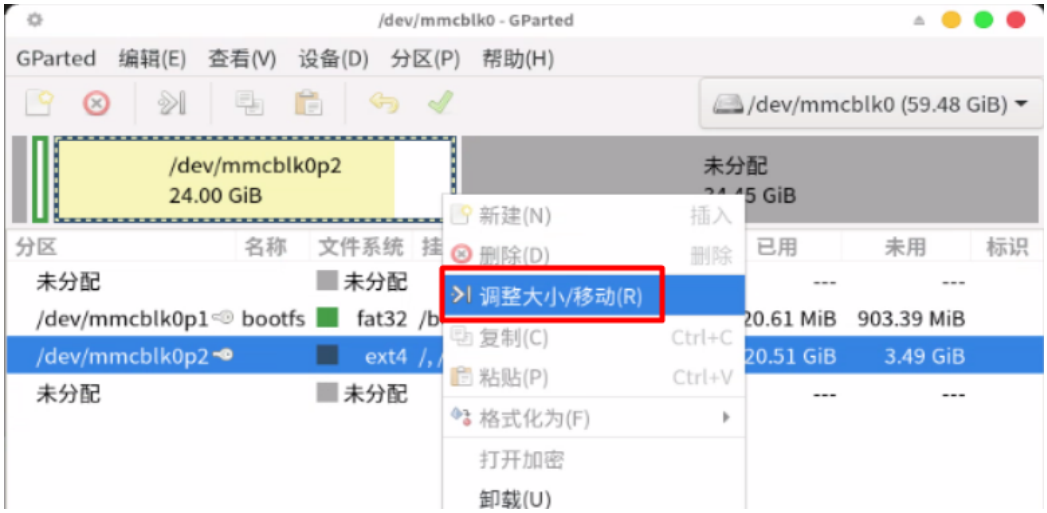
打开终端，输入

```
gparted
```

会弹出提示框，输入密码 orangepi，点击授权



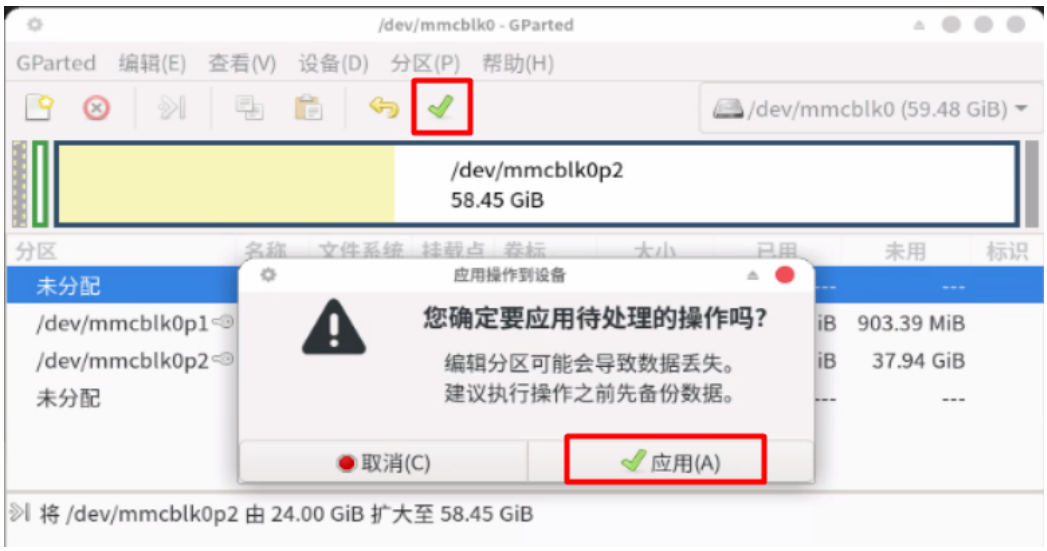
右击 /dev/mmcblk0p2 对应的部分，选择调整大小/移动



将右侧边界拉到最右侧，点击调整大小



点击 √，再点击应用



操作完成后点击关闭，再关闭 Gparted 窗口即可

此时系统盘空间得到了充分利用

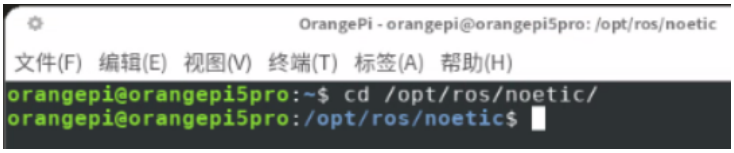
路径

打开终端后可以看到



图中框内即是当前终端的路径（ @ 前的 orangepi 为用户名，@ 后的 orangepi 为计算机名），在终端中进行的操作均是在该路径下的操作（右键打开终端的话，终端会自动进入右键点击地点的地址）

在终端中使用 cd 路径 指令可以进行路径跳转，例如



路径分为绝对路径和相对路径，上图即是通过绝对路径进行跳转

使用 cd ./ 来进入当前路径，./ 配合 Tab 键可以检索当前路径下的文件夹（按两下 Tab 为检索，按一下为补全名字），cd ./xxx 来进入下一级路径



可以使用 cd ../ 跳转到上一级路径



直接使用 cd ~/ 指令或 cd 可以跳转到主目录，主目录可以使用 ~/ 来表示，或者使用绝对路径 /home/用户名/ 表示



权限

如有需要使用 sudo 权限的时候，例如



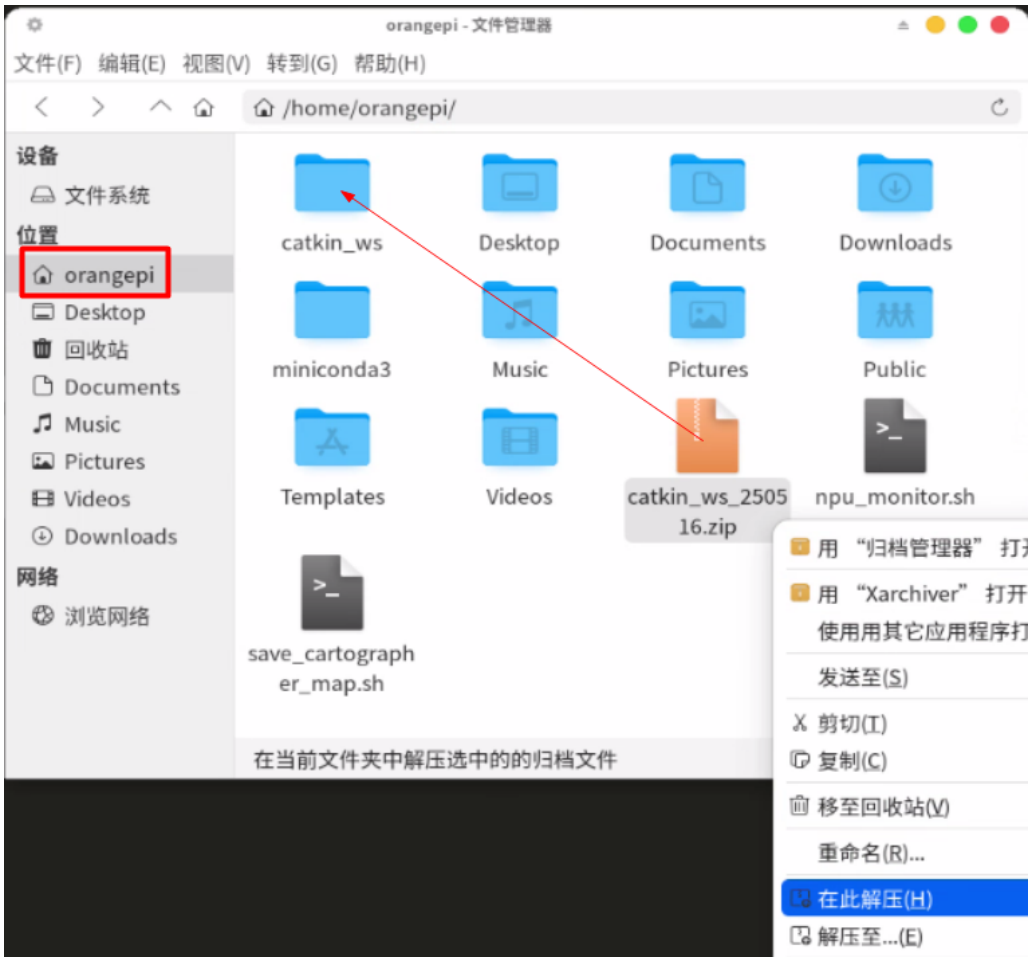
密码为 orangepi，输入密码的时候不会有显示，不要以为没有输入

文件传输

NoMachine 有文件传输功能，可传输单独的文件，不能传输文件夹，但可将文件夹压缩成压缩包传输

下载官方代码压缩包，将其拖动至远程连接窗口内，远程工具就会将压缩包传入 Orange Pi 5 Pro，默认传输至桌面上

须将代码压缩包转移至主目录 orangepi 中，然后将其解压



确保解压后的文件夹中只有这样三个文件夹，而不是有其它嵌套文件夹，否则后续会遇到路径错误



注释 / 取注 / 搜索

双击本项目所使用的任一源码文件，系统将默认使用 gedit 文本编辑器打开，在此编辑器中，选中某一或若干行，按 Ctrl + m 可注释掉选中的行，选中的行的代码即失效；同样地，在选中后按 Ctrl + Shift + m 可取消选中的行的注释，恢复选中的行的代码的运行

使用 VSCode 打开本项目时，代码的注释与取注快捷键为 Ctrl + /

Ctrl + f 可在当前打开的文本中打开搜索功能

切换输入法

在打开的文本或者终端中，按下 Ctrl + 空格，会切换输入法

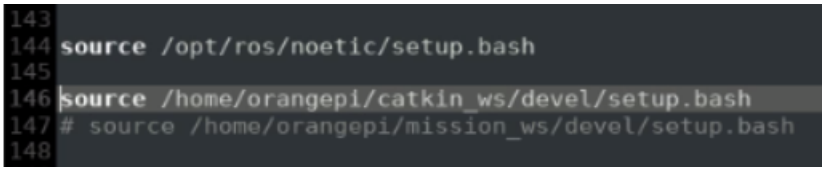


切换成功后，右上角会出现 “拼” 字样，即切换到拼音输入法，此时再按左 Shift 可切换中文/英文输入

环境变量修改

本项目使用 ROS，建议通过修改环境变量声明 ROS 工作空间位置

在主目录 orangepi 中，按 Ctrl + H，显示隐藏文件，双击打开 .bashrc，拖动到最下方



已预先写下了环境变量，取消注释即可，保存关闭

保存后，重新打开终端，相应的 ROS 工作空间即生效，可被系统识别到

结束终端运行的程序

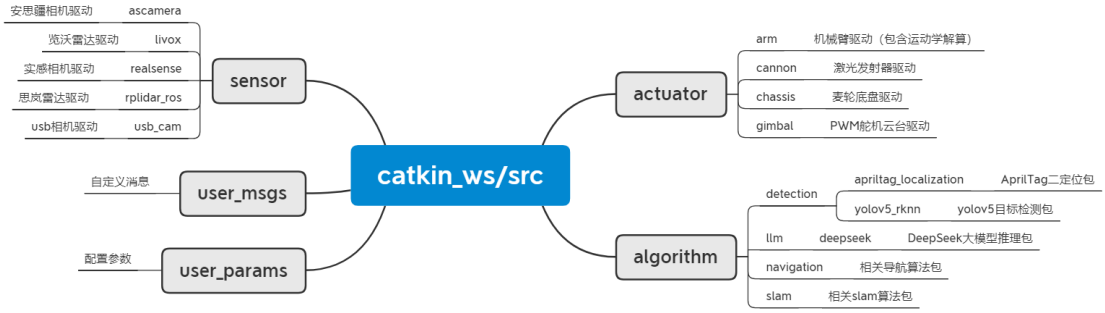
在终端中按下 Ctrl + c，可手动正常结束运行的程序

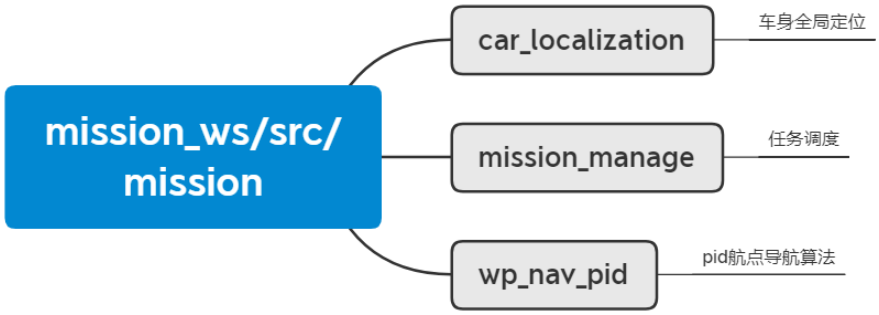
按下 Ctrl + z，可强制终止正在运行的程序

SubPart 3 代码架构简介

本项目涉及上位机对底盘与机械臂的控制，相机画面的获取，多种目标的检测，单目相机的空间定位，机械臂运动学的解算，以及相应的任务调度与决策

本项目使用 ROS 1 构建，将通用功能性功能包放置于 catkin_ws 工作空间中，将与本赛项相关的特化功能的功能包放置于 mission_ws 工作空间中





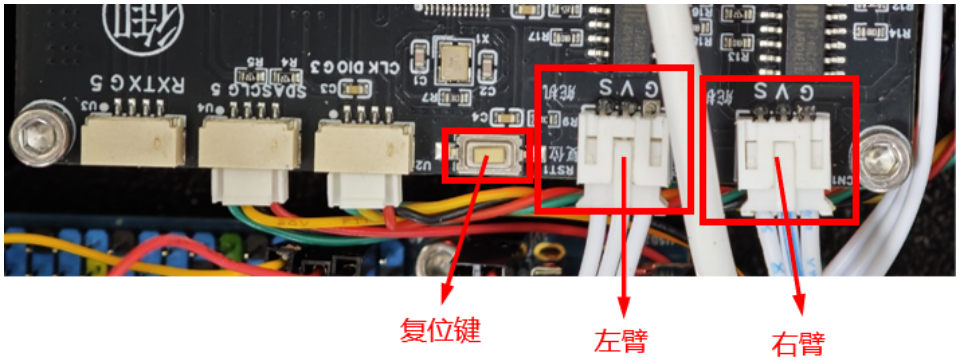
SubPart 4 机器人坐标系

坐标系	原点	x 轴正方向	y 轴正方向	z 轴正方向
地图坐标系	地图右下角	依据右手系得	向左	垂直地面向上
车体坐标系	机械臂立柱型材中心在地面上的投影	指向车头	指向车体左侧	沿立柱型材向上
机械臂坐标系	对应机械臂底座中心	底座舵机另一侧指向舵盘的方向	依据右手系得	沿机械臂底座向上
相机坐标系	rgb 相机光心	指向像素横坐标增长方向	指向像素纵坐标增长方向	依据右手系得
二维码坐标系	二维码图片中心	面向二维码，向里	面向二维码，向左	依据右手系得

Part 1 硬件校准测试

Step 1 校准机械臂

注意：如下图，机械臂上电即带有力矩，**如非程序关闭力矩请勿强力掰动机械臂，可能对舵机造成不可逆的损伤**



机械臂使用 I2C 进行通信，机械臂驱动板接入 Orange Pi 5 Pro 的 I2C-1，可使用如下指令查看 I2C 地址

```
sudo i2cdetect -y 1
```



其中 10 分配给左臂，11 分配给右臂

注意：如机械臂相关调试过程中遇到诸如角度读取异常的情况，通常表现为机械臂可控，但 RVIZ 中不随动，可尝试按复位键重置

先将 /home/orangepi/catkin_ws/src/user_params/params/arm_config.yaml 参数配置文件左/右机械臂中 1~5 号舵机的偏差量置 0，保存关闭

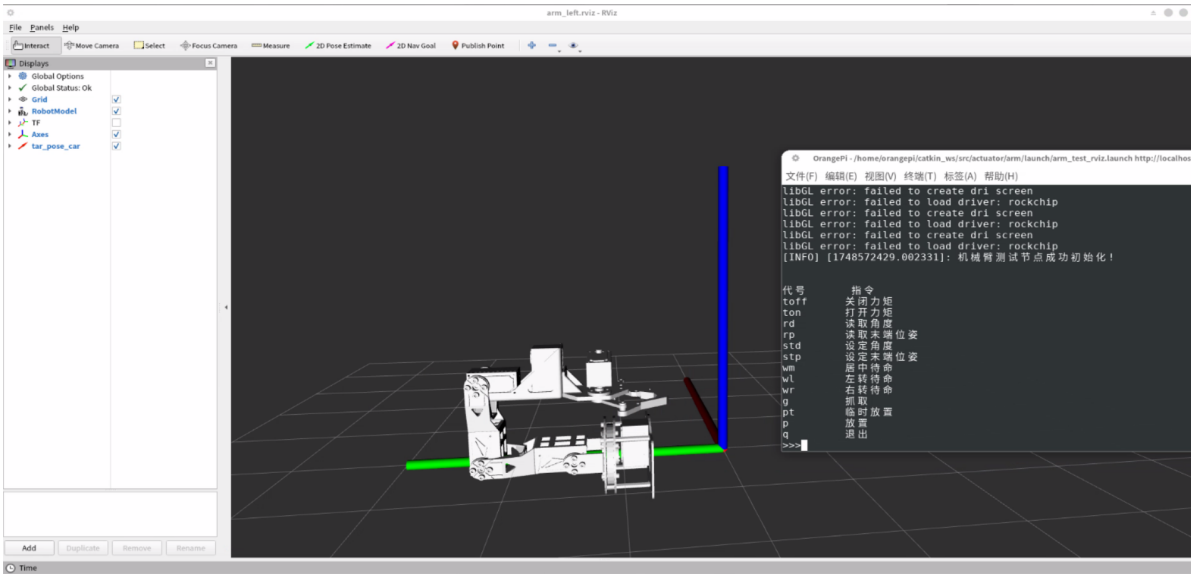
```
arm_left:
  delta_servo_1: 0
  delta_servo_2: 0
  delta_servo_3: 0
  delta_servo_4: 0
  delta_servo_5: 0
  delta_servo_6: 0

arm_right:
  delta_servo_1: 0
  delta_servo_2: 0
  delta_servo_3: 0
  delta_servo_4: 0
  delta_servo_5: 0
  delta_servo_6: 0
```

再运行机械臂调试

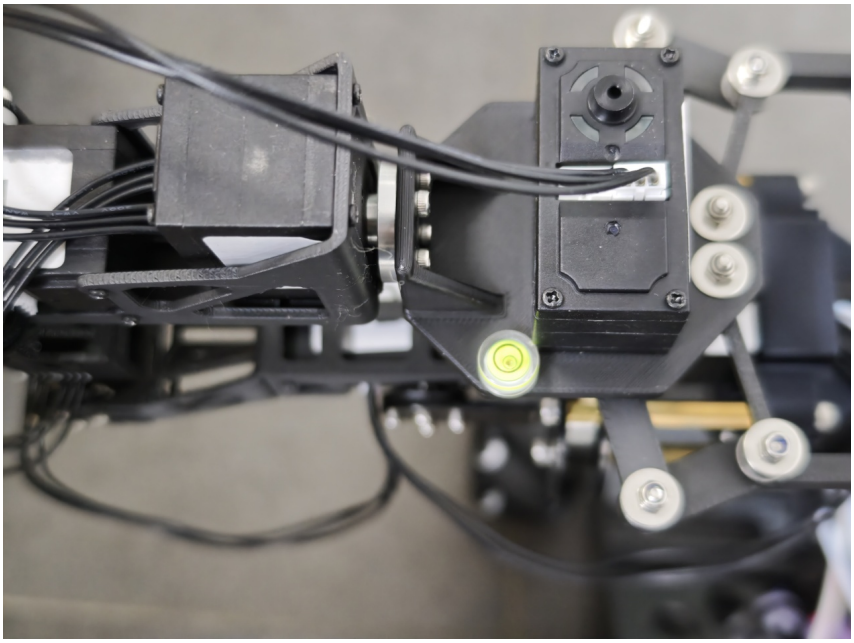
```
roslaunch arm arm_test_rviz.launch arm_dir:=left
或
roslaunch arm arm_test_rviz.launch arm_dir:=right
```

其中后面的 arm_dir:= 用来指定调试的机械臂，可使用 left 或者 right

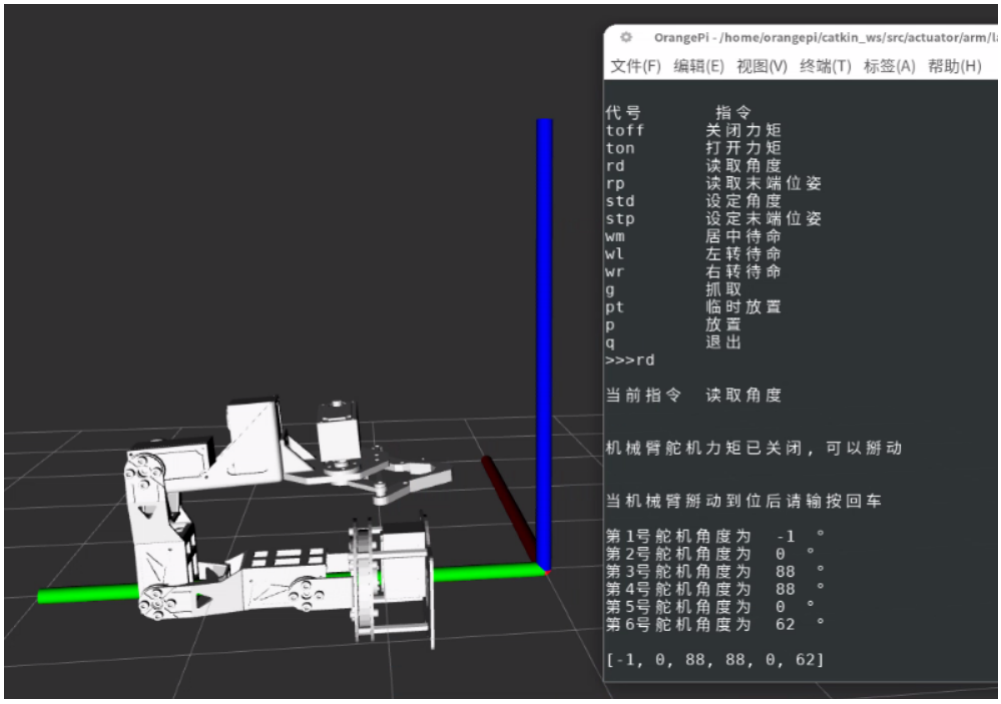


按终端提示输入设置角度指令 wm 并回车，此时机械臂将向上翻折如图，此时1-5 号舵机控制角度为 [0, 0, 90, 90, 0]

夹爪安装面应呈水平，如下图（建议配合水平仪测量）



若非水平，为确保后续夹取水果时坐标解算正确，需微调 1-5 号舵机的角度，使得相机支架底座水平。
按终端提示输入读取指令 rd 并回车，根据进一步的提示掰动机械臂



直到读取到水平时的各舵机角度，记录相应的舵机角度，并修改 arm_config.yaml 中相应舵机的偏差角度值（1-5 号舵机读取的角度减去控制角度[0, 0, 90, 90, 0]，即所需要的偏差量，以上图读取的结果为例，偏差量为 [-1, 0, -2, -2, 0]）

完成校准后，保存对 arm_config.yaml 的更改，退出正在运行的 launch 脚本，偏差量下次运行生效

测量机械臂外参

量取对应机械臂坐标系原点在车体坐标系中的位置，填入

```yaml
arm_left:
    rotation:     [0, 90, 90]  # car_2_arm
    translation:  [0.0, 0.075, 0.305]
arm_right:
    rotation:     [0, 90, -90]  # car_2_arm
    translation:  [0.0, -0.075, 0.305]
```

机械臂测试

运行机械臂调试，根据终端提示可测试后续会使用到的全部预置动作

Step 2 校准底盘

运行底盘校准脚本

```
cd /home/orangepi/catkin_ws/src/actuator/chassis/scripts
python3 chassis_calib.py
```



启动引擎后，先校准线速度系数，按提示输入指令 cl 并回车，小车会以半线速度前进 1s，测量这 1s 内前进的距离（m），将这个距离写到 /home/orangepi/catkin_ws/src/user_params/params/chassis_config.yaml 中的 l_0x40

接下来校准角速度系数，运行步骤同上，指令为 ca

小车会以半角速度自旋 1s，记录这 1s 内自旋的角度（°）（可以使用手机中的指南针功能配合测量角位移），将这个角度写到 chassis_config.yaml 中的 a_0x40

```
l_0x40:           0.50  # 线速度系数校准 1s内行驶的距离 m
a_0x40:           60.0  # 角速度系数校准 1s内旋转的角度 °
```

完成校准后，保存对 chassis_config.yaml 的更改，退出运行的代码

底盘死区特性测试

先将 chassis_config.yaml 的以下参数置零

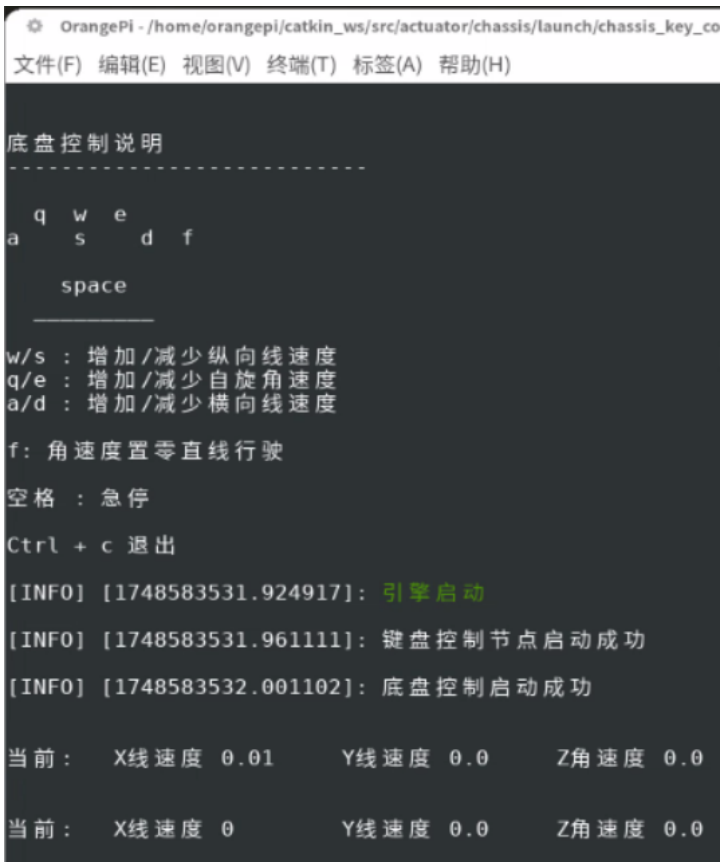
```
dead_zone_xy:     0.0  # 平移方向控制量小于该值则不执行 (可置零)
activate_zone_xy: 0.0  # 平移方向死区阈值 车子刚动起来的值
dead_zone_z:      0.0  # 自旋方向控制量小于该值则不执行 (可置零)
activate_zone_z:  0    # 自旋方向死区阈值 车子刚动起来的值
```

并将下来键盘控制阶跃参数改小（如下示意），保存关闭

```
key_control:  # 键盘控制
  step_size_linear:   0.01  # m/s
  step_size_angular:  1    # °/s
```

运行底盘键盘控制

```
roslaunch chassis chassis_key_control.launch
```



按一次键增加或减少相应轴的速度（不要长按某个键），分别测量出车刚刚能运动起来的线速度阈值和角速度阈值（默认使用正值），将其填写至 chassis_config.yaml 中对应的 xy_activate_zone 和 z_activate_zone，保存关闭

死区特性的实际作用代码如下

```
def threshold(x, dead_zone=0, activate_zone=0, clamp=0):
    if x > dead_zone:
        return min((x-dead_zone) + activate_zone, clamp)
    elif x < -dead_zone:
        return max((x+dead_zone) - activate_zone, -clamp)
    else:
        return 0
```

底盘获取原始的控制速度，然后使用上方函数，将小于 dead_zone 的控制速度置零，用于避免一直在运动，将大于 dead_zone 部分的控制速度加上 activate_zone，用于越过该轴的启动死区，确保能动起来，建议 activate_zone 应比实际能动起来的值小一些

Step 3 相机测试/校准

本项目使用 安思疆 Nuwa-HP60C 原深相机

启动相机

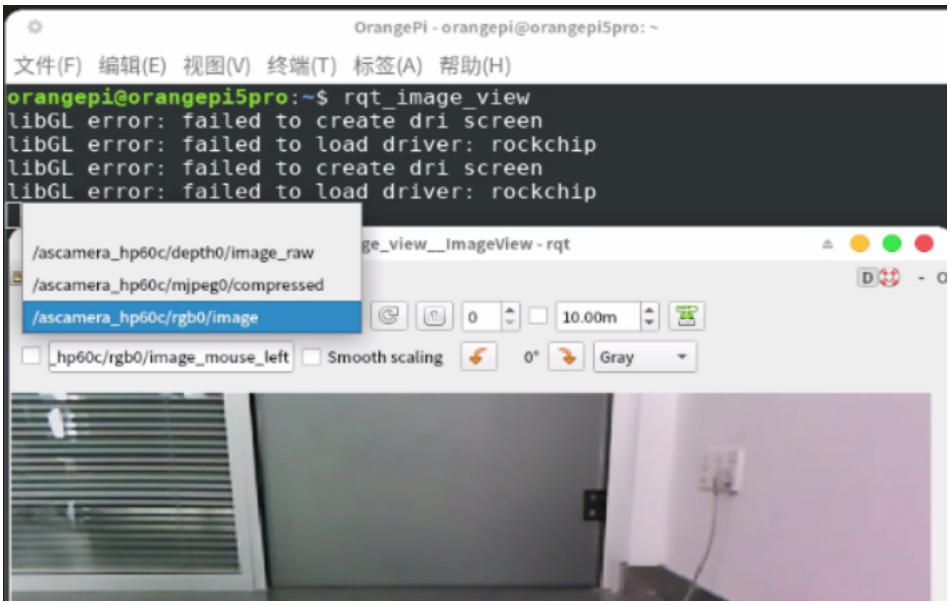
```
roslaunch ascamera hp60c.launch
```

该 launch 不启动任何显示界面，且无其它节点订阅图像话题的时候会自行关闭视频流

```
[ INFO] [1748584233.550308949]: camera opened
[ INFO] [1748584233.550488029]: get config info, ret 0, is_Registration 1
[ INFO] [1748584233.555882297]: #camera[0x557f90f2a0] SN[ASC60CE43000576]'s firm
ware version: DLS_VERc20865=:V01.09_2024110714:V01.19_2024110714:L01
[ INFO] [1748584233.557533973]: set depth resolution: 640x480@30fps
[ INFO] [1748584233.559699263]: set rgb resolution: 640x480@30fps
[ INFO] [1748584233.581440541]: 2025-05-30 13:50:33[INFO] [CameraHp60c.cpp] [259
] [startStreaming] start streaming
[ INFO] [1748584233.583390003]: attached end
[ INFO] [1748584234.600570956]: 2025-05-30 13:50:34[INFO] [CameraHp60c.cpp] [278
] [stopStreaming] stop streaming
```

打开一个新终端，运行测试

```
rqt_image_view
```



可切换图像话题名

相机内参已预先标定过，由相机本体提供，无需校准，但相机外参仍需标定，其获取方式见 Part 2 -> SubPart 1 二维码定位

Part 2 基础功能测试

SubPart 1 二维码定位

二维码定位的配置参数位于 /home/orangepi/catkin_ws/src/algorithm/detection/apriltag_localization/params/apriltag_localization_ascamera.yaml

```yaml
apriltag_localization:
  image_topic_name: /ascamera_hp60c/rgb0/image  # sub
  detected_topic_name: /apriltag_detected
  pose_topic_name: /cam_pose_tag                # pub

  tag_size: 0.133  # m
  id_list: [ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9,
             10, 11, 12, 13, 14, 15, 16, 17, 18, 19,
             20, 21, 22, 23, 24, 25, 26]

```

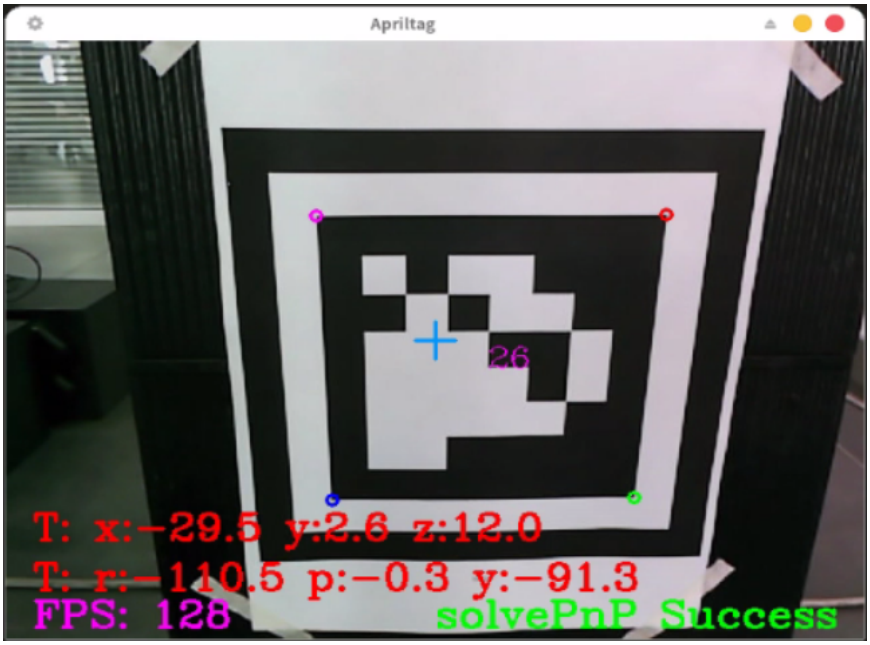
```
  resolve_method: 0
  # enum { SOLVEPNP_ITERATIVE = 0,
  #        SOLVEPNP_EPNP      = 1, //!< EPnP: Efficient Perspective-n-Point Camera Pose Estimation @cite lepetit2009epnp
  #        SOLVEPNP_P3P       = 2, //!< Complete Solution Classification for the Perspective-Three-Point Problem
  #        SOLVEPNP_DLS       = 3, //!< A Direct Least-Squares (DLS) Method for PnP  @cite hesch2011direct
  #        SOLVEPNP_UPNP      = 4, //!< Exhaustive Linearization for Robust Camera Pose and Focal Length Estimation
  #        SOLVEPNP_AP3P      = 5, //!< An Efficient Algebraic Solution to the Perspective-Three-Point Problem
  #        SOLVEPNP_MAX_COUNT      //!< Used for count
  #      };

  pub_tf: false
```

其中 tag_size 用于指定二维码的有效尺寸，id_list 用于指定二维码的有效 id 列表，只对列表内 id 的二维码进行位姿解算，防止误检测，resolve_method 用于指定解算算法，默认使用迭代法

开启二维码定位

```
roslaunch apriltag_localization apriltag_localization.launch
```



此节点实现计算相机坐标系在二维码坐标系中的位姿（位置xyz，姿态rpy），显示在图像上，只对参数文件 id_list 中指定 id 的二维码进行检测，且默认只对检测到的二维码中最靠近视野中心的那个二维码进行姿态解算

此外，还支持指令切换用于姿态解算的二维码 id

```
rostopic pub /at_target_id std_msgs/Int16 "data: 0"
```

其中 data 即期望的目标二维码 id，若视野中没有出现该二维码，则使用视野中心的二维码，若 data 为 -1，则同样默认只对检测到的二维码中最靠近视野中心的那个二维码进行姿态解算

相机外参标定

将车体放置在水平的地面上，将某张二维码贴在垂直于地面的平面上，调整车体位置，使得二维码出现在相机画面中，在画面中的占比尽量大，且相机视野中心（即画面中蓝色十字标）大致位于二维码水平中线上，且车头保持正对二维码所在平面，即保持车体坐标系与二维码坐标系的 x/y 轴都平行，类似上图，读取第二行数据

```
r:-110.5  p:-0.3  y:-91.3
```

将其填写至 /home/orangepi/catkin_ws/src/user_params/params/camera_ep_config.yaml 中的

```
ascamera:
    rotation:     [-110.5, -0.3, -91.3]  # °
```

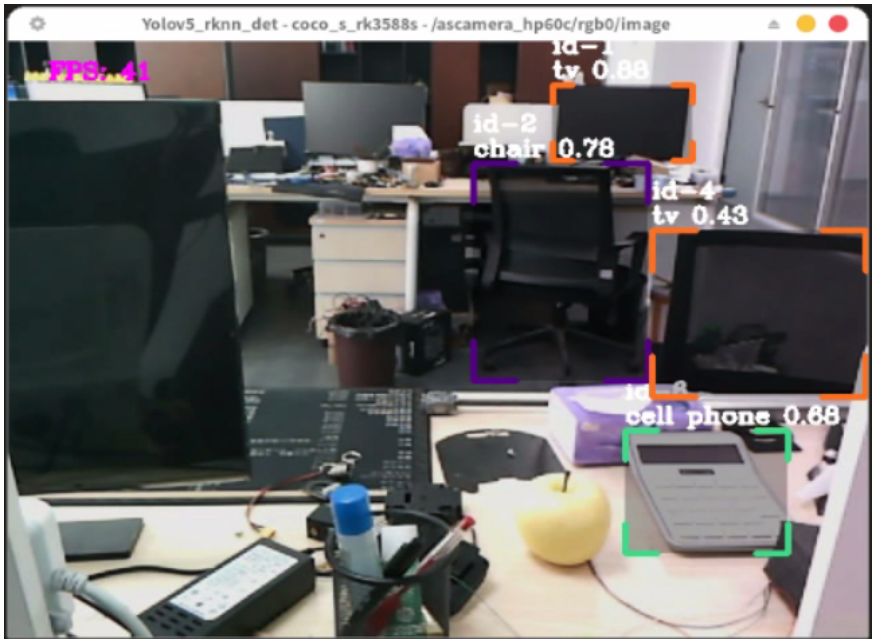
此外，使用测量器材，测量偏移向量，即相机坐标系原点在车体坐标系中的位置

```
ascamera:
    translation:  [0.035, 0.0, 0.415]  # m
```

SubPart 2 目标检测

开启目标检测

```
roslaunch yolov5_rknn yolov5_rknn_coco.launch
```



以上加载的是在 coco 数据集上训练的模型

自定义数据集训练/模型转换/部署

注意：本项目在 rk 官方提供的改进 yolov5 基础上实现，不同于 ultralytics 的 yolov5

Step 1 制作数据集

采集图片素材

/home/orangepi/catkin_ws/src/algorithm/detection/yolov5_rknn/scripts 中提供了简易的采集数据集与后期处理的脚本

运行

```
roslaunch yolov5_rknn prepare_dataset.launch
```

将显示相机画面，点击选中画面，如果确定采集当前素材，则按下空格，将自动保存当前画面至 /home/dataset/original/images 中

采集一定数量的场地元素（包括但不限于 6 种水果）照片（建议每种采集 100 张起，采集时需考虑如何提高数据集质量，数据集的质量会极大地影响训练的模型的效果）

采集结束后使用 labelimg 进行标注

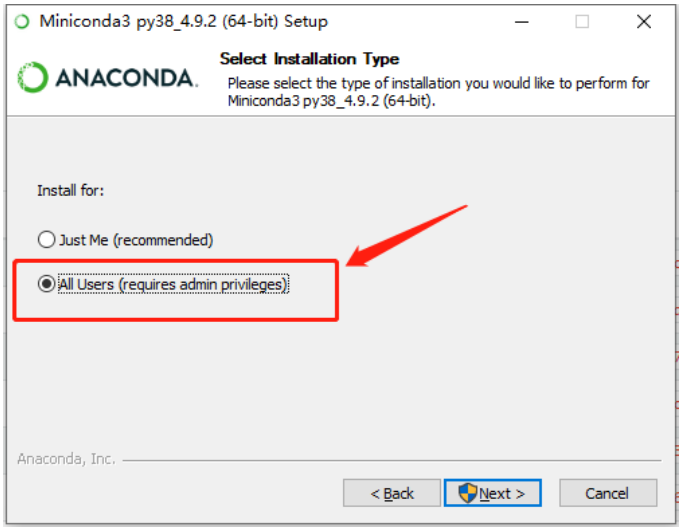
数据标注

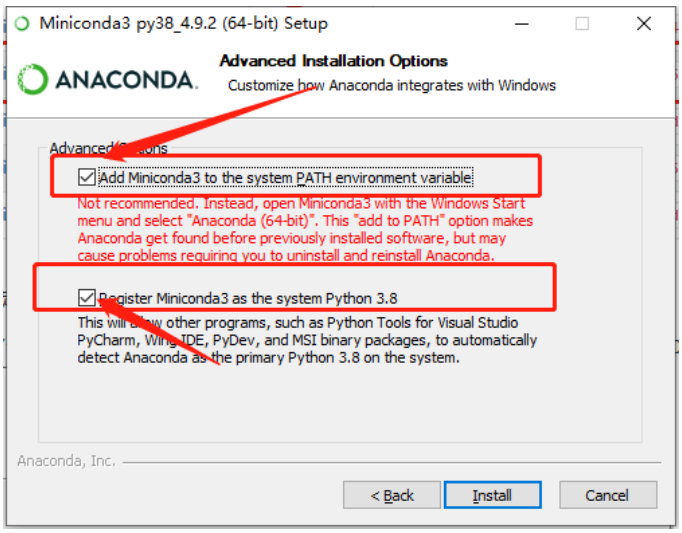
以下为 Windows 中的方案

SubStep 1 安装 conda

windows 若仅需进行包管理则安装 miniconda 更为方便 (https://docs.conda.io/en/latest/miniconda.html)

选择下载完毕之后右击安装包，以管理员权限安装即可，注意**安装路径**中**避免中文**，且这些一定要选中 (视版本不同，可能有些表述不完全一致，注意一定要勾选类似图二的添加环境变量)





安装完毕之后打开 windows 的命令行（win+r，输入 cmd，回车），输入 conda 回车，出现下列信息则表示 conda 已完成安装

```
(base) C:\Users\JIYU>conda
usage: conda-script.py [-h] [-V] command ...

conda is a tool for managing and deploying applications, environments and packages.

Options:

positional arguments:
  command
    clean              Remove unused packages and caches.
    compare            Compare packages between conda environments.
    config             Modify configuration values in .condarc. This is modeled after the git config command. Writes to
                       the user .condarc file (C:\Users\JIYU\.condarc) by default. Use the --show-sources flag to
                       display all identified configuration locations on your computer.
    create             Create a new conda environment from a list of specified packages.
    info               Display information about current conda install.
    init               Initialize conda for shell interaction.
    install            Installs a list of packages into a specified conda environment.
    list               List installed packages in a conda environment.
    package            Low-level conda package utility. (EXPERIMENTAL)
    remove (uninstall)
                       Remove a list of packages from a specified conda environment. Use `--all` flag to remove all
                       packages and the environment itself.
    rename             Renames an existing environment.
    run                Run an executable in a conda environment.
    search             Search for packages and display associated information. The input is a MatchSpec, a query
                       language for conda packages. See examples below.
    update (upgrade)   Updates conda packages to the latest compatible version.
    notices            Retrieves latest channel notifications.
```

注：若出现“‘conda’不是内部或外部命令，也不是可运行的程序”，则打开高级系统设置，检查环境变量，(可参考 https://blog.csdn.net/weixin_41957687/article/details/131349985)

安装完成后可切换国内源以提升后续库安装速度，在上述命令行输入（依次输入，复制一行，输入一行，回车，再继续）
注意：其中第一行指令可能会报错，因为无历史 key，不用管

```
conda config --remove-key channels
conda config --add channels
https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/free/
conda config --add channels
https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main/
conda config --set show_channel_urls yes

pip config set global.index-url https://mirrors.ustc.edu.cn/pypi/web/simple
pip config set install.trusted-host https://pypi.tuna.tsinghua.edu.cn
```

SubStep 2 安装 labelimg

打开 windows 的命令行（win+r，输入 cmd，回车）在命令行中输入下列指令创建虚拟环境

```
conda create -n 虚拟环境名(替换成自定义的英文名) python=3.x(所需python版本，例如3.8)
例: conda create -n labelimg python=3.8
```

中途需要按提示输入 Y，同意安装

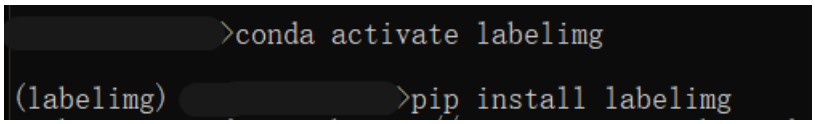
安装结束之后输入下列指令激活虚拟环境（关闭命令行即可取消激活，或命令行输入 conda deactivate）

```
conda activate 虚拟环境名(替换成自定义的英文名)
例: conda activate labelimg
```

虚拟环境激活成功后，在终端输入行的行首处将出现 (虚拟环境名)

安装labelimg，在激活了的虚拟环境下执行

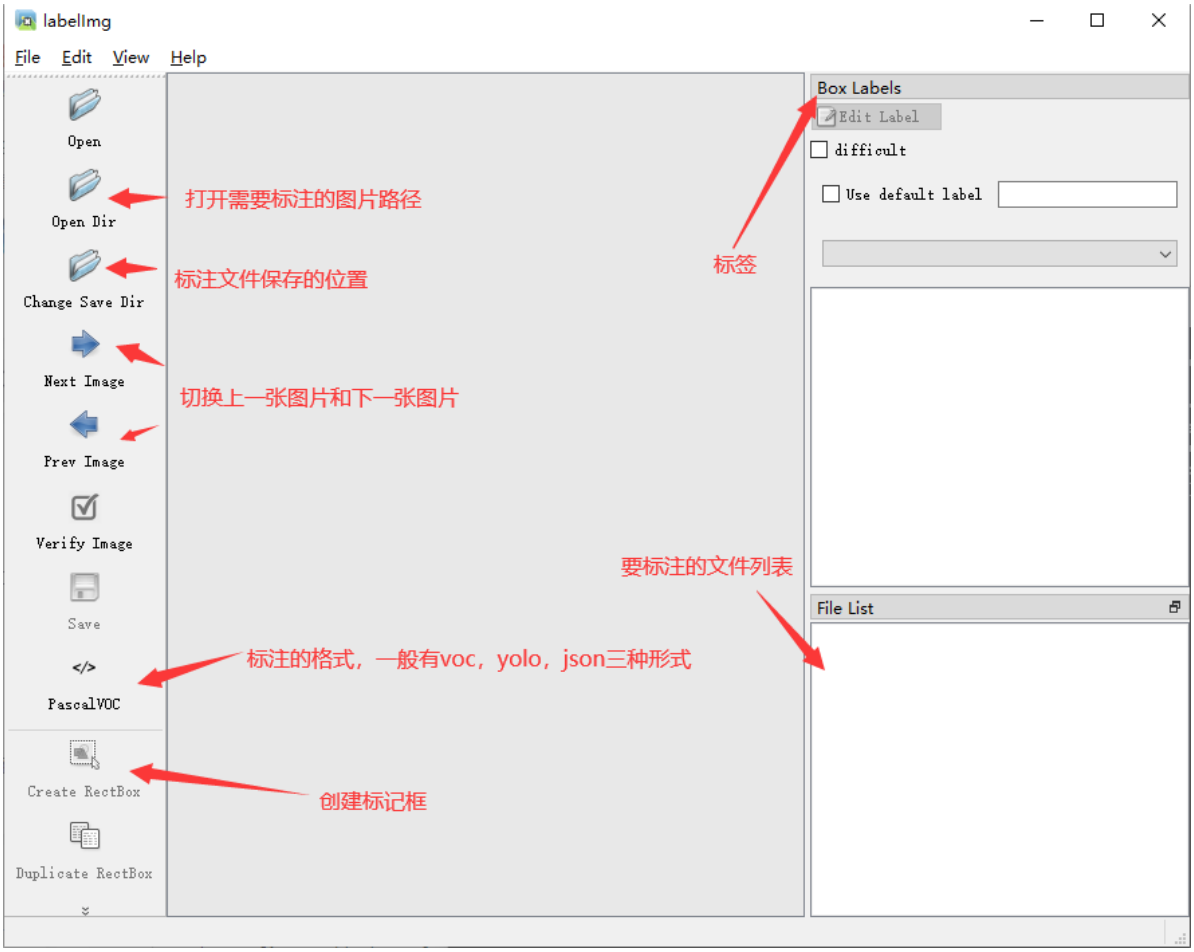
```
pip install labelimg
```



安装完成后在该虚拟环境下可打开标注界面，执行

```
labelimg
```

界面如下：



SubStep 3 使用 labelimg

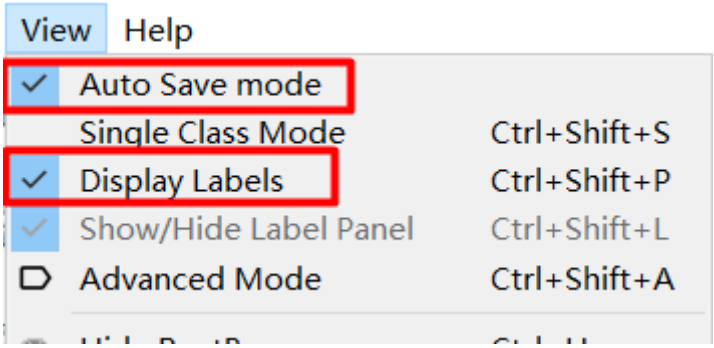
Open Dir 设置数据集图片路径，Change Save Dir 设置数据集标注文本路径



选择yolo格式



点击工具栏View，设置自动保存模式(该模式下切换图片时自动保存刚才标注图片的标注结果)与显示标签



创建检测框

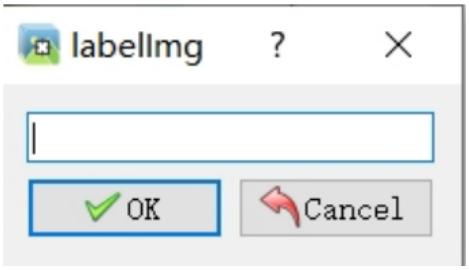


或使用快捷键 w

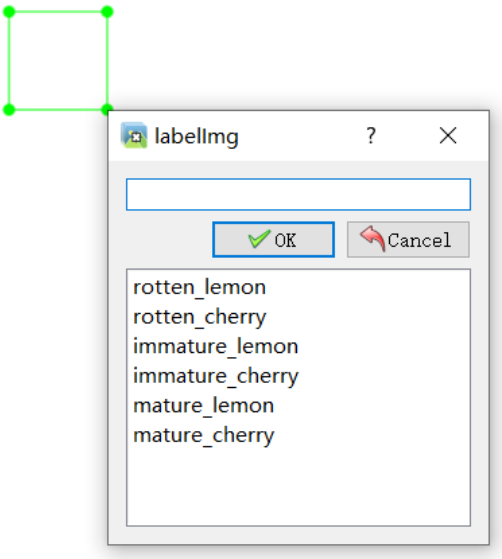
注：可能会遇到只能绘制正方形框，可点击工具栏Edit下的Draw Squares切换检测框模式

进入功能后，照片上会有十字标，按住鼠标将目标框出来，**使矩形尽量恰好包住目标**，如果不满意，选中后按Delete删除

创建标签

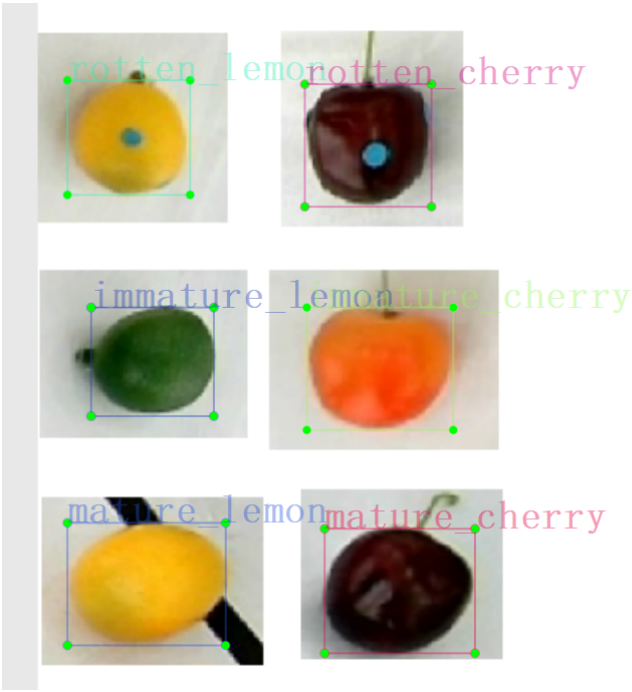


第一张会要输入标签的名字，后续同一种标签的就不用重新输入了



标注完成后会在之前设置的数据集标注文本路径下生成与图片同名的 .txt 标注结果（保存第一张图片时还会同时生成 classex.txt）

标注效果参考



上图及相应标注文件见 /home/orangepi/catkin_ws/src/algorithm/detection/yolov5_rknn/scripts/yolov5-rknn.zip 中的 dataset/original 下的 images 与 labels 中的 0.jpg 0.txt classes.txt

注意：labelimg不会自动加载标签文件，每次打开labelimg都需手动按顺序录入全部标签类型，否则会造成标签类型缺失错乱等问题。对此，可灵活使用上述 0.jpg 0.txt ，每次打开labelimg后手动按顺序移动一下每个标注框，全部移动后保存，即可实现全部录入

注意：视虚拟环境的 Python 版本不同，安装的不同版本的 labelimg 可能会在使用时遇到类似如下的一些问题

```
  File "                conda\envs\labelimg\lib\site-packages\libs\shape.py", line 131, in paint
    painter.drawText(min_x, min_y, self.label)
TypeError: arguments did not match any overloaded call:
  drawText(self, p: Union[QPointF, QPoint], s: Optional[str]): argument 1 has unexpected type 'float'
  drawText(self, rectangle: QRectF, flags: int, text: Optional[str]): argument 1 has unexpected type 'float'
  drawText(self, rectangle: QRect, flags: int, text: Optional[str]): argument 1 has unexpected type 'float'
  drawText(self, rectangle: QRectF, text: Optional[str], option: QTextOption = QTextOption()): argument 1 has unexpected
 type 'float'
  drawText(self, p: QPoint, s: Optional[str]): argument 1 has unexpected type 'float'
  drawText(self, x: int, y: int, width: int, height: int, flags: int, text: Optional[str]): argument 1 has unexpected ty
pe 'float'
  drawText(self, x: int, y: int, s: Optional[str]): argument 1 has unexpected type 'float'
```

一方面可以尝试换低版本的 Python，另一方面可以找到报错路径下的代码，按提示为报错的变量强制转整型，改如下 8 处

```
# \envs\labelimg\lib\site-packages\libs\shape.py", line 131, in paint
painter.drawText(min_x, min_y, self.label)
# 从
painter.drawText(min_x, min_y, self.label)
# 改为
painter.drawText(int(min_x), int(min_y), self.label)


# \envs\labelimg\lib\site-packages\labelImg\labelImg.py", line 965, in
scroll_request  bar.setValue(bar.value() + bar.singleStep() * units)
# 从
bar.setValue(bar.value() + bar.singleStep() * units)
# 改为
bar.setValue(int(bar.value()) + int(bar.singleStep() * units))

```

```
14
15  # \envs\labelimg\lib\site-packages\labelImg\labelImg.py", line 971, in
    set_zoom  self.zoom_widget.setValue(value)
16  # 从
17  self.zoom_widget.setValue(value)
18  # 改为
19  self.zoom_widget.setValue(int(value))
20
21  # \envs\labelimg\lib\site-packages\labelImg\labelImg.py", line 1025, in
    zoom_request  h_bar.setValue(new_h_bar_value)
22  # 从
23  h_bar.setValue(new_h_bar_value)
24  # 改为
25  h_bar.setValue(int(new_h_bar_value))
26
27
28  # \envs\labelimg\lib\site-packages\labelImg\labelImg.py", line 1026, in
    zoom_request  h_bar.setValue(new_v_bar_value)
29  # 从
30  h_bar.setValue(new_v_bar_value)
31  # 改为
32  h_bar.setValue(int(new_v_bar_value))
33
34
35  # \envs\labelimg\lib\site-packages\libs\canvas.py", line 530, in paintEvent
    p.drawLine(self.prev_point.x(), 0, self.prev_point.x(),
    self.pixmap.height())
36  # 从
37  p.drawLine(self.prev_point.x(), 0, self.prev_point.x(),
    self.pixmap.height())
38  # 改为
39  p.drawLine(int(self.prev_point.x()), 0, int(self.prev_point.x()),
    int(self.pixmap.height()))
40
41
42  # \envs\labelimg\lib\site-packages\libs\canvas.py", line 531, in paintEvent
    p.drawLine(0, self.prev_point.y(), self.pixmap.width(), self.prev_point.y())
43  # 从
44  p.drawLine(0, self.prev_point.y(), self.pixmap.width(), self.prev_point.y())
45  # 改为
46  p.drawLine(0, int(self.prev_point.y()), int(self.pixmap.width()),
    int(self.prev_point.y()))
47
48
49  # \envs\labelimg\lib\site-packages\libs\canvas.py", line 526, in paintEvent
    p.drawRect(left_top.x(), left_top.y(), rect_width, rect_height)
50  # 从
51  p.drawRect(left_top.x(), left_top.y(), rect_width, rect_height)
52  # 改为
53  p.drawRect(int(left_top.x()), int(left_top.y()), int(rect_width),
    int(rect_height))
```

快捷键

按键	功能
w	创建检测框
a	上一张图
d	下一张图
Delete	删除选中的检测框
Ctrl + s	保存
Ctrl + e	编辑标签

Step 2 训练

将 /home/orangepi/catkin_ws/src/algorithm/detection/yolov5_rknn/scripts/yolov5-rknn.zip 传递至 PC，解压

PC 中的训练环境需自行配置（如需使用 GPU 加速，不建议使用虚拟机），在此不赘述

修改 yolov5-rknn/yolov5-relu/models 文件夹下的 yolov5_ .yaml （yolov5_ .yaml中的 _ 需对应自己模型的尺度 [n, s, m, l, x]，在 OrangePi 5 Pro 上可以使用 s 模型，即在 yolov5s.yaml 中修改），修改 nc 为自己数据集标注的分类数量

```
nc: 80  # number of classes
```

将标注好的数据集划分为 训练/验证/测试 集
使用 /home/orangepi/catkin_ws/src/algorithm/detection/yolov5_rknn/scripts/dataset_split.py，修改其中的数据集路径

```
src_data_folder = "/home/orangepi/dataset/original/"   # todo 修改你的原始数据集路径
target_data_folder = "/home/orangepi/dataset/split/"  # todo 修改为你要存放的路径
```

运行

```
# 在 /home/orangepi/catkin_ws/src/algorithm/detection/yolov5_rknn/scripts 路径下
python3 dataset_split.py
```

划分好数据集后，修改 yolov5-rknn/yolov5-relu/data/user_data.yaml 中的数据集相关信息，例如

```yaml
# 训练集，验证集，测试集路径
train: (your_path_to)/dataset/split/images/train
val: (your_path_to)/dataset/split/images/val
test: (your_path_to)/dataset/split/images/test

# 分类数量
nc: 6

# 分类名称
names: ['r_l', 'r_c',
        'im_l', 'im_c',
        'm_l', 'm_c']
```

修改 yolov5-rknn/yolov5-relu/models/yolo.py 中的 forward 函数
注意：训练时，取注训练部分相关代码，注释掉转换部分相关代码

以下指令中的 data 为数据集配置文件（即上文的 user_data.yaml），cfg 为使用的模型的配置文件（即上文的 yolov5_ .yaml ），weights 为预训练的模型权重（一般从 weight 文件夹中官方的预训练模型权重起开始训练），epoch 为训练轮数（小分类数量时建议100起，若数据集标注效果好的话此时已基本收敛），batch-size 为一次训练的样本数（习惯上为 2 的幂，且不建议太大），视设备性能自行尝试选取

```
# 在 yolov5-rknn/yolov5-relu 路径下运行
python3 train.py --data user_data.yaml --cfg yolov5s.yaml --weights
weights/yolov5s.pt --epoch 100 --batch-size 16
```

训练结束后将会在 yolov5-relu/runs/train/exp_ 中得到相应的 last.pt 和 best.pt 文件，得到相应的 last.pt 和 best.pt 文件后即完成了训练，可查看该文件夹中生成的图表自行评估训练效果

Step 3 模型转换

以下需在 x86_64 的 linux 系统中进行（x64_64架构常见于各种 PC，也可在 PC 上使用虚拟机）
yolov5-rknn 压缩包中提供了 rknn_toolkit2-1.5.2+b642f30c-cp38-cp38-linux_x86_64.whl 和 requirements_cp38-1.5.2.txt，需自行在 x86_64 的 linux 系统中配置环境
建议使用 miniconda，自行安装并创建python3.8的虚拟环境

```
pip3 install -r requirements_cp38-1.5.2.txt
pip3 install rknn_toolkit2-1.5.2+b642f30c-cp38-cp38-linux_x86_64.whl
```

将 yolov5-rknn 传入 x86_64 的 linux 系统

SubStep 1 pt -> onnx

修改 yolov5-rknn/yolov5-relu/models/yolo.py 中的 forward 函数
注意：转换时，取注转换部分相关代码，注释掉训练部分相关代码

将训练结束后生成的 best.pt 复制到 yolov5-rknn/yolov5-relu 路径下

```
# 在 yolov5-rknn/yolov5-relu 路径下运行
python3 export.py --rknpu --weights best.pt
```

该路径下将会生成 best.onnx 和 RK_anchors.txt

SubStep 2 onnx -> rknn

将生成的 best.onnx 复制到 yolov5-rknn/yolov5_onnx_2_rknn 路径下

修改 yolov5-rknn/yolov5_onnx_2_rknn/test.py 中的相关部分，自行搜索关键字找到对应行进行修改

```
ONNX_MODEL = 'yolov5s_relu.onnx'
RKNN_MODEL = 'yolov5s_relu.rknn'
IMG_PATH = './bus.jpg'
DATASET = './dataset.txt'

CLASSES = ("person", "bicycle", "car", "motorbike", "aeroplane", "bus", "train", "truck", "boat", "traffic light", "fire hydrant", "stop sign", "parking meter", "bench", "bird", "cat", "dog", "horse ", "sheep", "cow", "elephant", "bear", "zebra", "giraffe", "backpack", "umbrella", "handbag", "tie", "suitcase", "frisbee", "skis", "snowboard", "sports ball", "kite", "baseball bat", "baseball glove", "skateboard", "surfboard", "tennis racket", "bottle", "wine glass", "cup", "fork", "knife", "spoon", "bowl", "banana", "apple", "sandwich", "orange", "broccoli", "carrot", "hot dog", "pizza", "donut", "cake", "chair", "sofa", "pottedplant", "bed", "diningtable", "toilet", "tvmonitor", "laptop", "mouse", "remote", "keyboard", "cell phone", "microwave", "oven", "toaster", "sink", "refrigerator", "book", "clock", "vase", "scissors", "teddy bear", "hair drier", "toothbrush")

anchors = [[10, 13], [16, 30], [33, 23], [30, 61], [62, 45],
           [59, 119], [116, 90], [156, 198], [373, 326]]

rknn.config(mean_values=[[0, 0, 0]], std_values=[[255, 255, 255]], target_platform='rk3566')

```

其中

变量	修改方式
ONNX_MODEL	用于转换的 onnx 文件名，改成 yolov5-rknn/yolov5_onnx_2_rknn 下的 best.onnx
RKNN_MODEL	生成的 rknn 文件名
IMG_PATH	yolov5-rknn/yolov5_onnx_2_rknn 路径下用于推理测试的图片名
DATASET	yolov5-rknn/yolov5_onnx_2_rknn 路径下的 dataset.txt，需修改该 txt 文件中的内容
CLASSES	模型检测的种类名称
anchors	SubStep 1 pt -> onnx 中的 RK_anchors.txt 中的值，按顺序写入
target_platform	目标开发板芯片型号，将 rk3566 改成本项目使用的 rk3588s

运行

```
# 在 yolov5-rknn/yolov5_onnx_2_rknn 路径下运行
python3 test.py
```

运行结束后会在 yolov5-rknn/yolov5_onnx_2_rknn 路径下生成 rknn 文件以及测试图片检测结果图

Step 4 部署

准备好生成的 rknn 文件、RK_anchors.txt 以及检测种类名

将 rknn 文件传入 OrangePi 5 Pro，将其替换掉 /home/orangepi/Documents/yolov5_weights/rc_s_rk3588s.rknn，同时修改同一路径下的 labels_rc.txt 中的种类名称，注意第一行的数字为种类数量，用于程序判断种类名称是否填写完全以免出错

修改 /home/orangepi/catkin_ws/src/algorithm/detection/yolov5_rknn/params/yolov5_rc_rk3588s_ascamera.yaml

```
yolov5_rknn_rc:
  image_topic_name: /ascamera_hp60c/rgb0/image

  model_url: /home/orangepi/Documents/yolov5_weights/rc_s_rk3588s.rknn
  labels_url: /home/orangepi/Documents/yolov5_weights/labels_rc.txt

  class_num: 6
  anchors: [10, 13, 16, 30, 33, 23, 30, 61, 62, 45, 59, 119, 116, 90, 156, 198, 373, 326]

  conf_thresh: 0.5
  iou_thresh: 0.5

  kill_switch_topic: /task

  byte_track:
    target_class_id: 0
    track_thresh: 0.4
    high_thresh: 0.5
    match_thresh: 0.7
    track_buffer: 60
```

其中 yolov5_rknn_rc 用于和相应 launch 文件中的命名空间同步，修改 model_url、labels_url、class_num、anchors 等

运行

```
roslaunch yolov5_rknn yolov5_rknn_rc.launch
```

SubPart 3 大模型

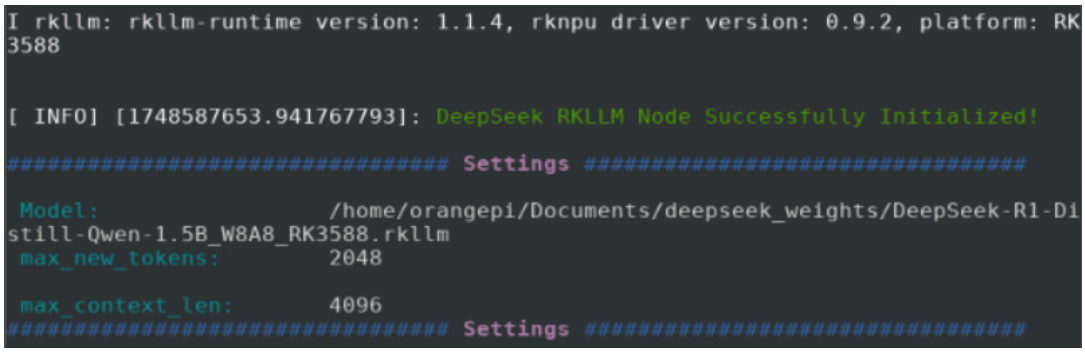
大模型的配置参数位于
/home/orangepi/catkin_ws/src/algorithm/llm/deepseek/params/deepseek_config.yaml

```yaml
deepseek:
  model_url: /home/orangepi/Documents/deepseek_weights/DeepSeek-R1-Distill-Qwen-1.5B_W8A8_RK3588.rkllm
  # model_url: /home/orangepi/Documents/deepseek_weights/DeepSeek-R1-Distill-Qwen-1.5B_FP16_RK3588.rkllm
  max_new_tokens: 2048
  max_context_len: 4096
  temperature: 0.8
  top_p: 0.95
  top_k: 1
  repeat_penalty: 1.1
```

其中的 model_url 指定模型路径，系统镜像中预留了 rk 官方的 1.5B 参数量的 W8A8 和 FP16 量化的两个模型

```
roslaunch deepseek deepseek.launch
```

加载模型需要一定的时间，加载完成后会显示当前设置

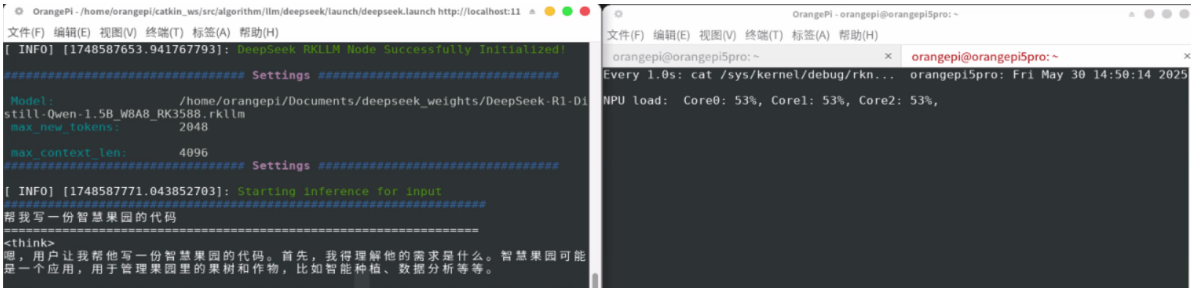


打开一个新终端，测试时使用 rostopic pub 功能输入测试文本

```
rostopic pub /deepseek/input_text std_msgs/String "data: '帮我写一份智慧果园的代码'"
```

大模型接收到输入后进行推理时会使用到 npu，在主目录中预留了 npu 监测脚本，运行后会显示 npu 使用情况

```
./npu_monitor.sh
```



在此基础上，可以使用 “自定义附加提示词” + “比赛指令” 的文本形式，将文本输入 deepseek 节点，进行推理

其中自定义附加提示词用于为大模型描述比赛任务背景与要求，以及规定输出形式
比赛指令将由类似 “将熟柠檬放到红色放置框中...最后停到 1 号车位” 组成

不断优化自定义附加提示词，期望大模型最终输出类似下方的结果，以便于后续任务的处理与执行

```
[[坏柠檬，红色]，[熟柠檬，黄色]，[坏樱桃，蓝色]，[熟樱桃，绿色]，[1]]
```

Part 3 综合功能测试

SubPart 1 放置区测序

考虑在出发区中，相机画面即可大致覆盖放置区，可以将车体放置在一个可重复放置的固定位置，然后采集四个放置框的 ROI，订阅相机画面，依据 ROI 中的像素值判断对应位置的放置框是什么颜色

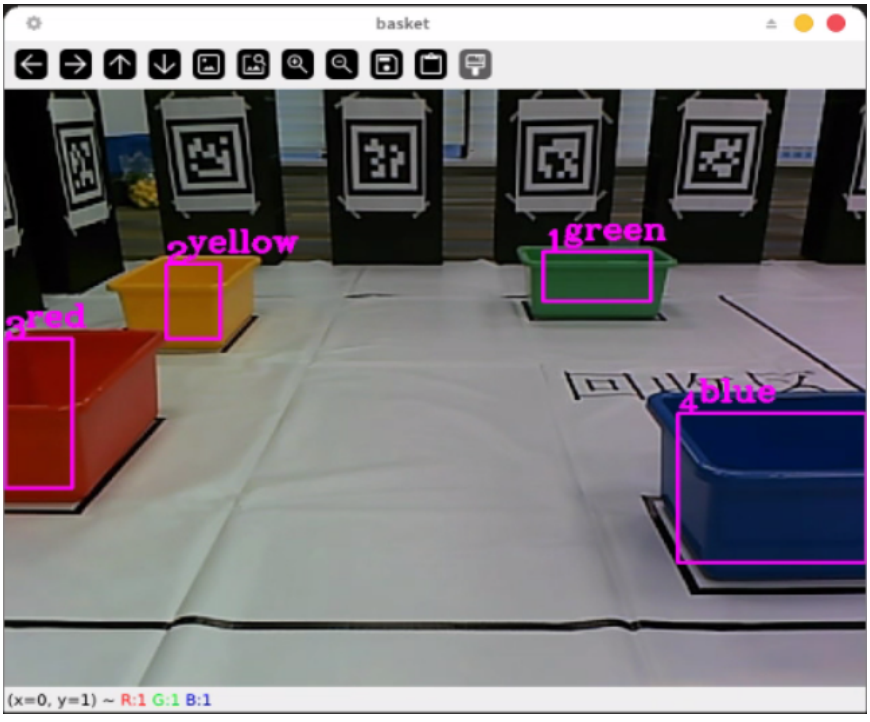
修改 /home/orangepi/mission_ws/src/mission/mission_manage/params/mission_params.yaml 中的放置框 ROI 角点位置

```
mission_magage:
    sort_basket_rois: [[[400, 130], [480, 170]],  # 1号框t/br
                       [[120, 140], [160, 200]],  # 2号框t/br
                       [[0, 200],   [50, 320]],   # 3号框t/br
                       [[500, 260], [640, 380]]]  # 4号框t/br
```

运行

```
roslaunch mission_manage basket_seq_test.launch
```

效果如下



终端中会打印类似如下的列表

```
[3, 1, 0, 2]
```

列表中对应位置的数字即该位置的放置框的颜色对应的代号

颜色与数字代号的映射关系在
/home/orangepi/mission_ws/src/mission/mission_manage/scripts/utils.py 中定义

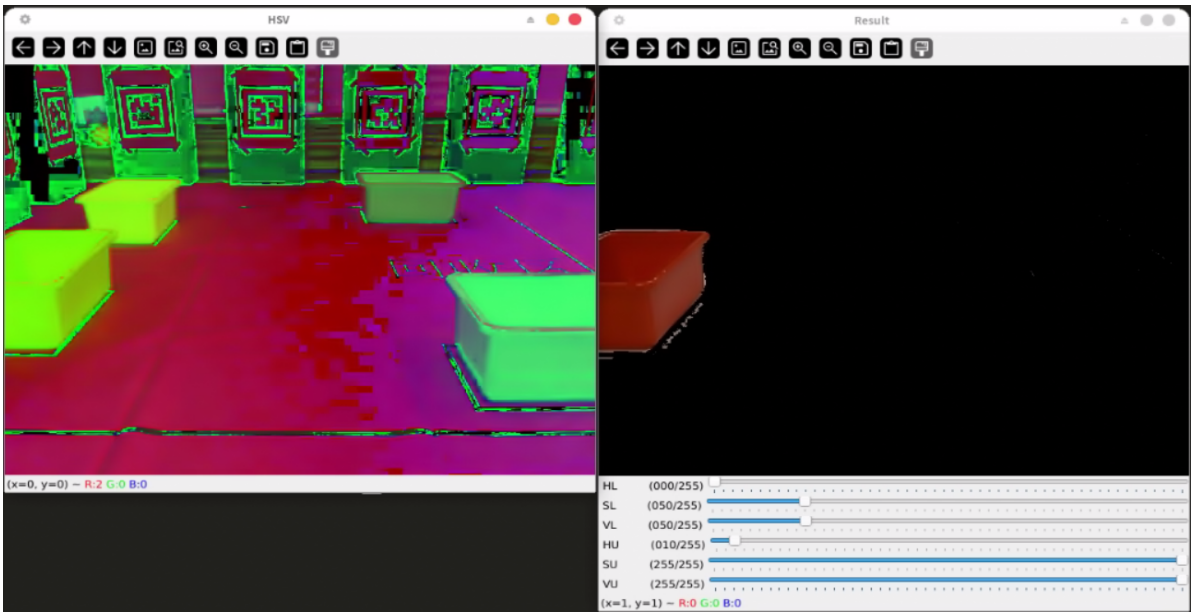
```
color_dict = {
    "红":   0,
    "黄":   1,
    "蓝":   2,
    "绿":   3
}
```

如发现检测结果不对应，则可修改 utils.py 中的 HSV 颜色阈值

```
# "color_name": ([lower_threshold], [upper_threshold], color_id),
color_info = {
    "red1":   ([0, 50, 50], [5, 255, 255], 0),
    "red2":   ([175, 50, 50], [180, 255, 255], 0),
    "yellow": ([20, 50, 50], [30, 255, 255], 1),
    "blue":   ([90, 50, 50], [120, 255, 255], 2),
    "green":  ([40, 50, 50], [80, 255, 255], 3)
}
```

运行

```
roslaunch mission_manage basket_hsv_threshold.launch
```



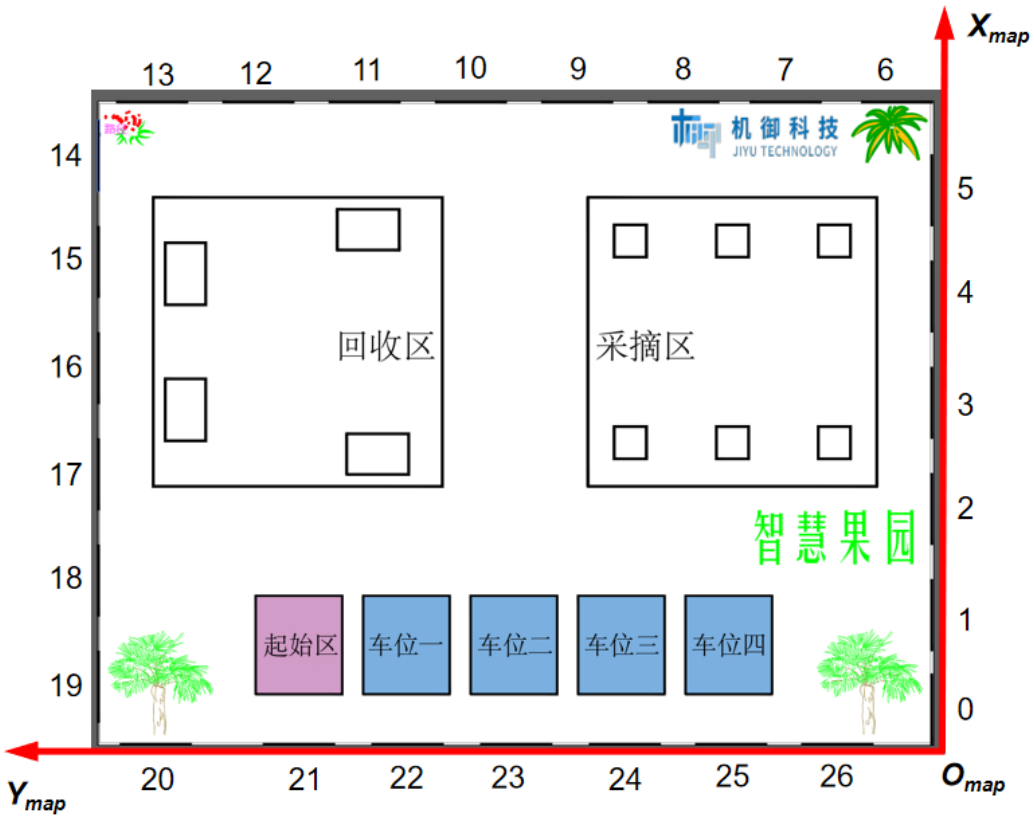
调节 Result 窗口中的滑块，使得 Result 中只保留一种颜色，点击任意画面按 Esc 退出，终端中将打印最后调节的阈值，复制到 color_info 中该颜色对应的部分

SubPart 2 二维码定位导航

在 Part 2 -> SubPart 1 二维码定位中，已实现了相机坐标系在二维码坐标系中的位姿解算，考虑到环场地二维码阵列中每一个二维码相对场地的位置已经固定，且相机在车体中的位姿也固定，依据位姿转换的链式传导，可解算车体在场地中的位姿

$$
\begin{aligned}
T_{car2map} &= T_{car2tag}T_{tag2map} \\
&= T_{car2cam}T_{cam2tag}T_{tag2map}
\end{aligned}
$$

首先需获得二维码阵列中每一个二维码对应的坐标系在场地坐标系中的位姿



在 /home/orangepi/mission_ws/src/mission/car_localization/params/tag_pose.yaml 中已提供了一个，可视实际测量情况自行调整

```yaml
car_localization:
  tag_pose:
    id_list: [ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9,
               10, 11, 12, 13, 14, 15, 16, 17, 18, 19,
               20, 21, 22, 23, 24, 25, 26]

    tag_0: [0.150, 0.0, 0.3, 0.0, 0.0, -90.0]  # [x, y, z, R, P, Y]  m °
    tag_1: [0.545, 0.0, 0.3, 0.0, 0.0, -90.0]
    tag_2: [0.990, 0.0, 0.3, 0.0, 0.0, -90.0]
    tag_3: [1.435, 0.0, 0.3, 0.0, 0.0, -90.0]
    tag_4: [1.880, 0.0, 0.3, 0.0, 0.0, -90.0]
    tag_5: [2.325, 0.0, 0.3, 0.0, 0.0, -90.0]

    tag_6: [2.7, 0.150, 0.3, 0.0, 0.0, 0.0]
    tag_7: [2.7, 0.617, 0.1, 0.0, 0.0, 0.0]
    tag_8: [2.7, 1.045, 0.1, 0.0, 0.0, 0.0]
    tag_9: [2.7, 1.490, 0.1, 0.0, 0.0, 0.0]
    tag_10: [2.7, 1.935, 0.3, 0.0, 0.0, 0.0]
    tag_11: [2.7, 2.380, 0.3, 0.0, 0.0, 0.0]
```

```
    tag_12: [2.7, 2.825, 0.3, 0.0, 0.0, 0.0]
    tag_13: [2.7, 3.270, 0.3, 0.0, 0.0, 0.0]

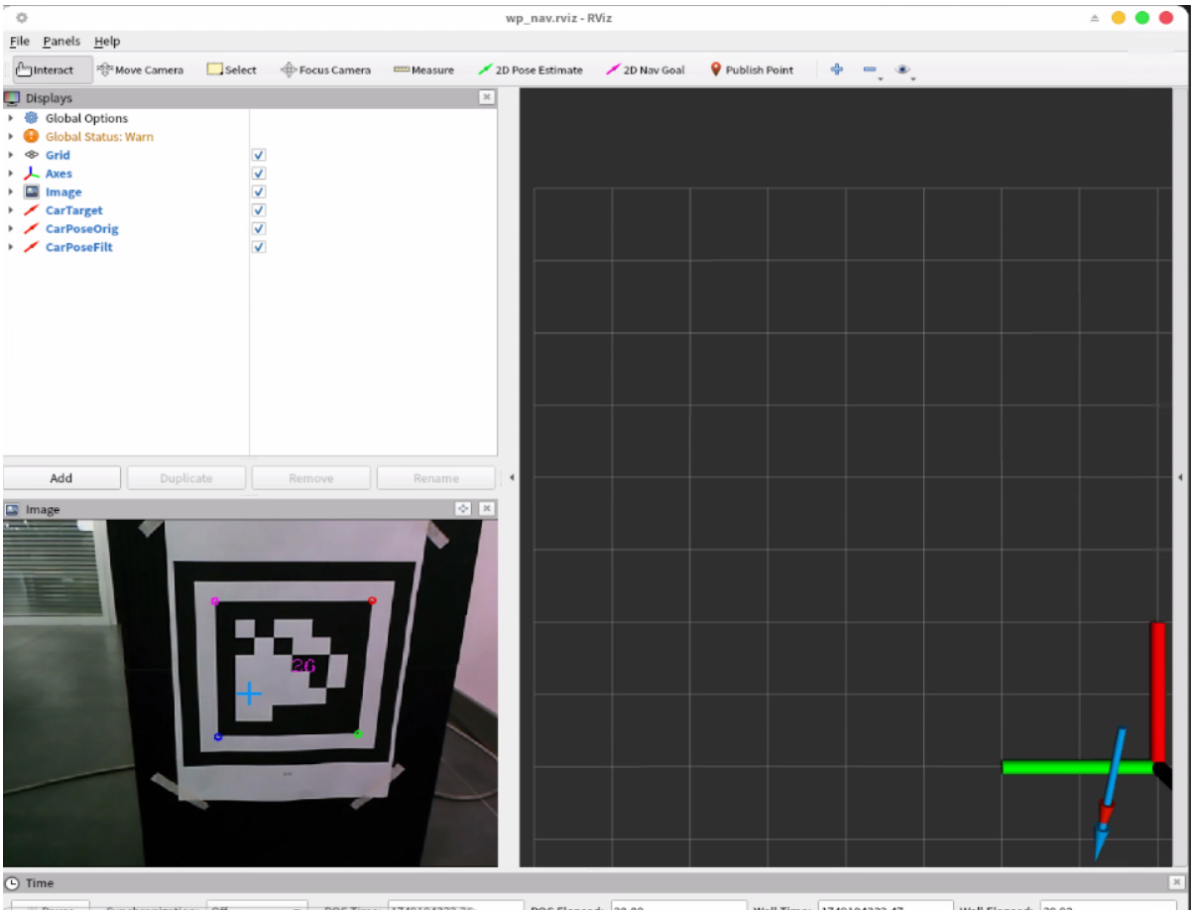
    tag_14: [2.470, 3.5, 0.3, 0.0, 0.0, 90.0]
    tag_15: [2.025, 3.5, 0.3, 0.0, 0.0, 90.0]
    tag_16: [1.580, 3.5, 0.3, 0.0, 0.0, 90.0]
    tag_17: [1.135, 3.5, 0.3, 0.0, 0.0, 90.0]
    tag_18: [0.690, 3.5, 0.3, 0.0, 0.0, 90.0]
    tag_19: [0.245, 3.5, 0.3, 0.0, 0.0, 90.0]

    tag_20: [0.0, 3.255, 0.3, 0.0, 0.0, 180.0]
    tag_21: [0.0, 2.655, 0.3, 0.0, 0.0, 180.0]
    tag_22: [0.0, 2.200, 0.3, 0.0, 0.0, 180.0]
    tag_23: [0.0, 1.745, 0.3, 0.0, 0.0, 180.0]
    tag_24: [0.0, 1.290, 0.3, 0.0, 0.0, 180.0]
    tag_25: [0.0, 0.845, 0.3, 0.0, 0.0, 180.0]
    tag_26: [0.0, 0.400, 0.3, 0.0, 0.0, 180.0]
```

运行

```
roslaunch car_localization car_localization.launch
```

可在 RVIZ 中可视化地显示车体位姿

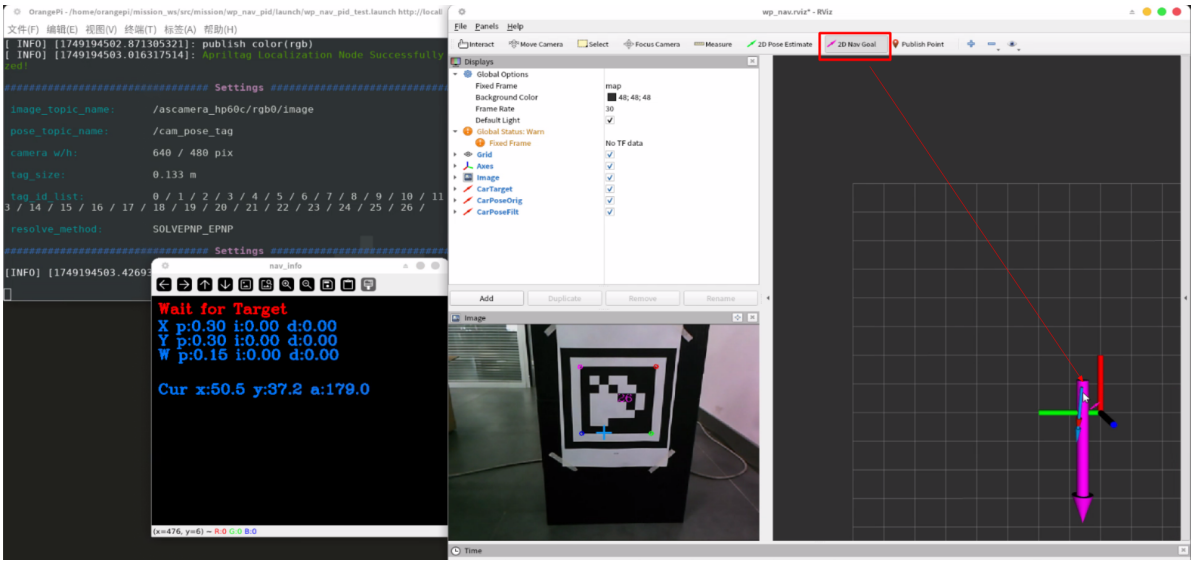


其中红色箭头为原始位姿，蓝色箭头为滤波后的位姿

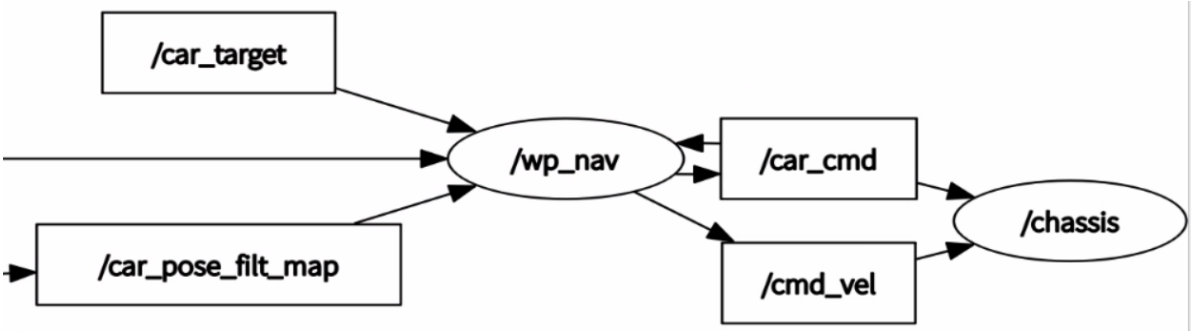
导航

结束之前运行的定位，重新运行

```
roslaunch wp_nav_pid wp_nav_pid_test.launch
```



可使用 RVIZ 中的 2D Nav Goal 功能，在图中点击左键，拖动紫红色箭头，发布导航目标点，小车会尝试导航到目标点



目前的导航代码中，如果导航中途丢失二维码，则将失去定位信息，小车会自行后退并自旋，增大视野以尝试重新获得定位

需调试 PID 参数，位于 /home/orangepi/mission_ws/src/mission/wp_nav_pid/params/wp_nav_pid.yaml

```yaml
wp_nav:
  pid_x:
    kp: 0.3
    ki: 0
    kd: 0
    i_a: 0
    o_t: 0.3
    tune_range: [1.0, 0.1, 1.0, 0.8, 1.0]  # pid调节器范围

  pid_y:
    kp: 0.3
    ki: 0
    kd: 0
    i_a: 0
    o_t: 0.3
    tune_range: [1.0, 0.1, 2.0, 0.8, 1.0]  # pid调节器范围

  pid_yaw:
    kp: 0.4
```

```
    ki: 0
    kd: 0
    i_a: 0
    o_t: 30
    tune_range: [2.0, 0.1, 2.0, 40, 60]  # pid调节器范围

  exit_l: 0.02  # m
  exit_a: 2  # °
  exit_cnt: 5000
```

其中，kp, ki, kd 即该轴的 pid 系数，i_a 为积分控制的作用域，该轴误差小于 i_a 时启用积分项，减少积分饱和现象，o_t 为输出限幅，tune_range 为 pid 调参窗口每个参数的可调节上限数值
exit_l 为默认 x/y 位置到位精度，exit_a 为默认 a 角度到位精度，exit_cnt 为默认连续到位次数 (此处默认 5000，意在调参时只用发布一个目标点，可以反复地移开小车让其导航回位)

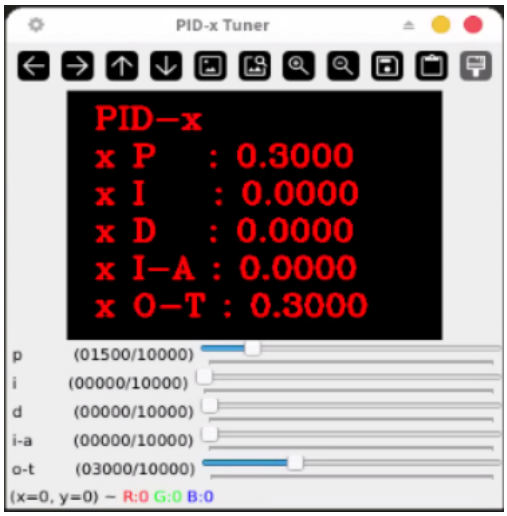
在 wp_nav_pid_test.launch 中设置了开启在线整定 PID 的参数，默认关闭调参，调试导航 PID 参数时，可以一轴一轴地进行单独调试（打开 tune，并打开所需调试的轴所对应的开关）

```
<arg name="tune" default="false" />
<arg name="tune_x" default="false" />
<arg name="tune_y" default="false" />
<arg name="tune_yaw" default="false" />
```

可以在 launch 文件中手动修改，或者在 roslaunch 指令中手动开启（下例为单独调试 x 轴）

```
roslaunch wp_nav_pid wp_nav_pid_test.launch tune:=true tune_x:=true
tune_y:=false tune_yaw:=false
```

运行时，可以将小车放置得靠近二维码一些，发布新目标点，然后调节滑块，改善运动性能



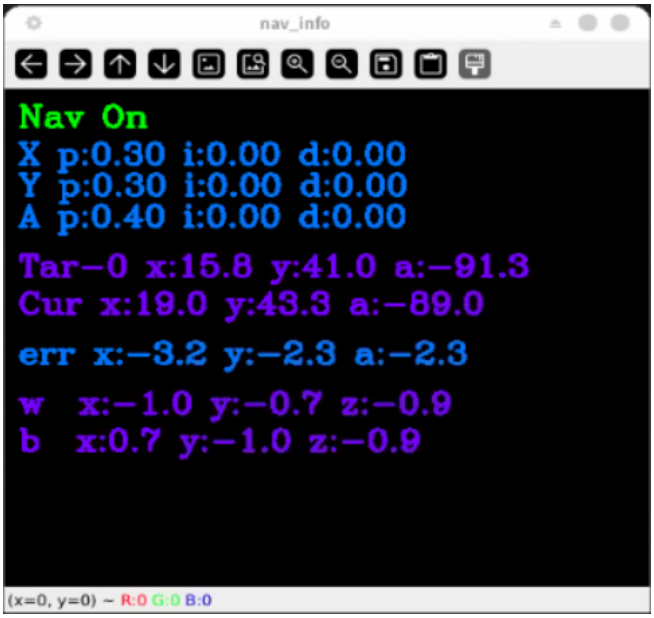
直到能快速稳定地到达目标点（终端打印连续到位），点击 Tuner 界面，按 Esc 键，终端将打印类似下方的字样，即修改的 PID 参数，将其复制保存到 PID 参数文件中

```
#############################################################

  pid_x:
    kp: 0.3
    ki: 0.0
    kd: 0.0
    i_a: 0.0
    o_t: 0.3
    tune_range: [2.0, 0.5, 2.0, 0.8, 1.0]  # pid调节器范围
```

调参技巧:

关注导航信息窗口



导航窗口中包含:

行数	内容
1	导航节点 状态
2	X 轴 线速度 PID
3	Y 轴 线速度 PID
4	Z 轴 角速度 PID
5	当前目标航点信息
6	当前车体位姿信息
7	**三轴误差**
8	全局/地图坐标系 三轴控制量
9	**车体坐标系 三轴控制量**

导航时关注第 7 / 9 行数据

第 9 行 车体坐标系三轴控制量 /cmd_vel 发给底盘节点，底盘节点接收后向底盘单片机发送速度指令，控制电机转动
第 7 行 连续一定次数误差小于设定的到位条件后才能发布到位状态的底盘状态 /car_cmd

调节导航参数时，由于车体重量较大，从静摩擦到动摩擦的跳变较大，还需协同调节底盘的死区特性以及每个轴的导航 PID，死区特性说明详见 Part 1 -> Step 2 校准底盘

整定 pid 参数的时候通常将 i 和 d 置零，先调节 p，使其能动起来，且在该轴上不至于震荡，p 不建议太大
然后调节积分控制 i (注意，需先调节 i_a，否则 i_a 默认为 0 时，不会启用积分控制)，启用积分控制后，积分的控制量会叠加到输出控制量上，只要存在误差，误差就会一直累积，积分控制量就会一直增大，故 i 不建议大，如加入积分控制后出现震荡，则考虑改小该轴死区特性的 activate_zone，并适当增加一些 dead_zone，让底盘节点过滤掉微小控制量，不运动
还需注意积分作用域 i_a 的选取，当改小 activate_zone 后，存在积分作用域外单纯依靠 p 控制的控制量在底盘节点叠加 activate_zone 后无法达到让车动起来的值的情况，而此时积分控制尚未开启，则将进入不可控区间，需修改积分作用区间，使得积分作用区间覆盖这样的不可控区间，但积分作用区间不建议太大，否则过早开启积分控制，误差一直累加会导致输出控制量过大而容易震荡
d 控制也可加入，可以进一步抑制震荡

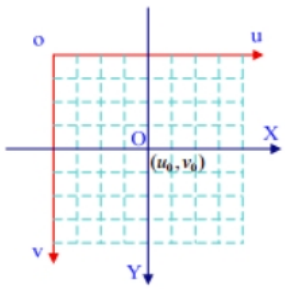
SubPart 3 水果采摘

Step 0 空间定位原理

为实现机械臂抓取水果，需知道目标在机械臂坐标系中的空间位置，这样就能调用机械臂运动学反解库，解算出各舵机的角度，控制舵机转动到对应角度，实现抓取

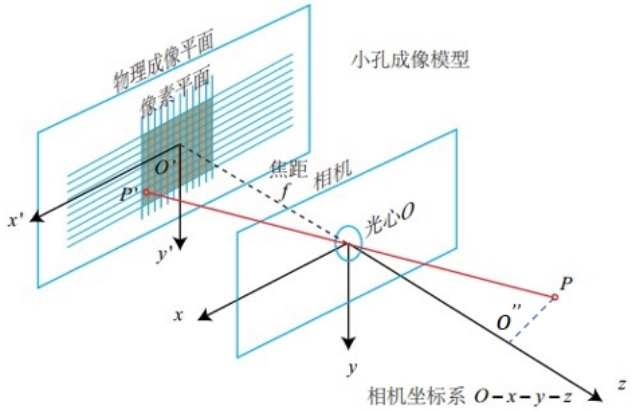
上述的目标检测可以获取目标中心在像素平面中的坐标，区别于 Part 3 中使用二维码与 PnP 解算实现的定位，受水果在树上的吊装方式的限制，无法找到先验的空间点位置，故无法进行 PnP 解算。为实现2D点到3D点的解算，此处使用单目相机基于内参的深度估计，其核心是小孔成像模型的相机中的相似性变换，该部分原理同样可以参考《视觉SLAM十四讲：从理论到实践》

1. 首先在物理成像平面上解算摄像头像素坐标系→相平面坐标系：像素坐标系原点固连在像平面的左上角 *o* 点，即 0 行 0 列的像素点处，为得到后续物体在相机坐标系下的位置，需将像素坐标系原点移动到镜头光轴在像素坐标系中的投影 *O* 处（代码中表现为 (cx, cy)）



$$\begin{bmatrix} X_p \\ Y_p \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} U \\ V \end{bmatrix} - \begin{bmatrix} c_x \\ c_y \end{bmatrix}$$

2. 然后解算相平面坐标系→相机坐标系



以像平面坐标系 *X* 轴方向为例，由图可知，$\triangle OO'P' \backsim \triangle OO''P$，故有

$$\frac{O''P}{O'P' = X_p \cdot dx} = \frac{OO''}{OO' = f}$$

其中 *X·dx* 为目标点在相平面坐标系内的 *x* 坐标，表征目标点距像平面原点距离为 *X* 个像素点宽度，*dx* 为像素点宽度，*f* 为相机焦距。通过相机标定，可获得相机焦距 *f* 与单个像素点的物理尺寸 *dx* 的比值，为常量（代码中表现为 fx 与 fy ），则有

$$X_c = O''P = \frac{X_p \cdot OO''}{\frac{f}{dx}}$$

若另知物体到相机坐标系平面的距离 *OO''*（即相机坐标系下的 *Z* 轴值），则可以通过相似变换，求得物体在 *X* 轴的投影坐标值

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = Z_c \times \begin{bmatrix} \frac{1}{f_x} & 0 & 0 \\ 0 & \frac{1}{f_y} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_p \\ Y_p \\ 1 \end{bmatrix}$$

需注意，本产品使用单目相机，常规方法无法获取精确的深度信息（即距离 *OO''*），但是在项目任务中，可以让小车导航到果树前的固定点位，由于果树在地图中的位置是固定的，可认为果子到车体坐标系的距离已知，故 *OO''* 可知

3. 接下来解相机坐标系->车体坐标系

已知的是 *Xb*，而需要的是*Xc*，通过相机到车体坐标系的外参可以反解得到*Xc*

$$\begin{bmatrix} X_b \\ Y_b \\ Z_b \end{bmatrix} = R_{c2b} \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} + t_{c2b}$$

有

$$X_b = r_{11}X_c + r_{12}Y_c + r_{13}Z_c + t_x$$

则有

$$Z_c = \frac{X_b - t_x}{r_{11}X_p + r_{12}Y_p + r_{13}}$$

得到相机坐标系下的目标位置后，只需再做一次旋转平移即可得到车体坐标系中的位置

$$\begin{bmatrix} X_b \\ Y_b \\ Z_b \end{bmatrix} = R_{c2b} \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} + t_{c2b}$$

4. 最后解算引入末端夹爪后机械臂5号舵机舵盘中心在底座坐标系下的位置

   由于常根据不同的使用场景使用不同的末端执行器，一般来说，机械臂运动学逆解求得是机械臂末端关节的位姿，并不是末端执行器（夹爪，吸盘等）的，故需进行这一步解算

   确定目标点在车体坐标系下与原点的偏移角度，根据目标高度确定夹爪俯仰角度，引入夹爪长度，根据预设的姿态角，计算偏移量更新 *xyz*

注意：此时获得的是车体坐标系中的 J5 末端位姿，为了实现机械臂运动学逆解，还需转换到对应机械臂的机械臂坐标系中，这里使用 ROS 中的 tf 功能进行转换

Step 1 抓取测试

首先测量 夹紧/松开 水果时 J6 的角度，使用 Part 1 -> Step 1 中的机械臂测试的读取角度功能，待其关闭力矩后，将水果放入夹爪，手动夹紧，读取此时 J6 的角度，将其填入 /home/orangepi/catkin_ws/src/user_params/params/arm_config.yaml 中的

```yaml
arm_left:
    claw_tight: 30
    claw_loose: 60
```

选定一个基准位置，记录此时二维码全局定位的位姿，量取不同位置水果到车体坐标系的 x 距离，并记录到 /home/orangepi/mission_ws/src/mission/mission_manage/params/mission_params.yaml 中

```yaml
grab_param:
    x_j5_2_claw: 0.085  # m
    z_j5_2_claw: 0.0115  # m

    sight_middle_h:         300  # pix
    sight_middle_w:         [269, 457]  # pix
    x_obj_to_cam_close:     0.205  # m
    x_obj_to_cam_middle:    0.26  # m
    x_obj_to_cam_far:       0.34  # m
```

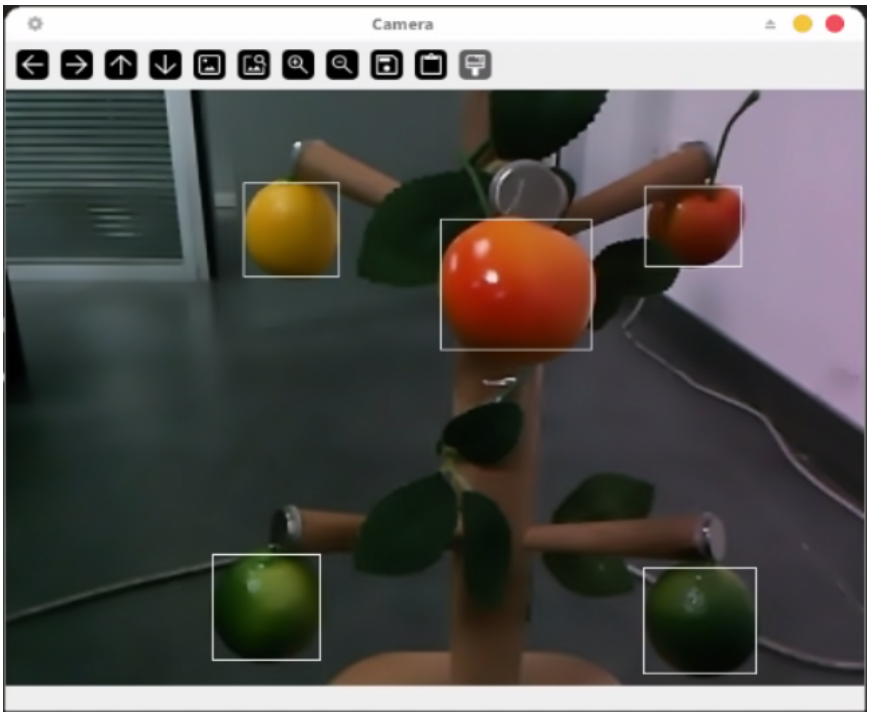
其中，视野中心阈值 以及 远/中/近 三档距离对应的水果方位可参考 /home/orangepi/mission_ws/src/mission/mission_manage/scripts/utils.py 代码中 calc_j5_from_claw 判断调用部分（**注意，下方判断并不能完全分割全部画面，存在抓取盲区，故对于小车到达的抓取位置的精度要求较高**）

```python
# 根据像素位置判断物体距离(因每次停止位置变动很小而可以认为距离为定值)
    if p_pixel[1] <= g_p.sight_middle_h and g_p.sight_middle_w[0] <= p_pixel[0] <= g_p.sight_middle_w[1]:
        x_obj_2_base = g_p.x_obj_to_cam_close + err_x
        print(f"上层中间果到相机CCD距离校正: {g_p.x_obj_to_cam_close} + {err_x} = {x_obj_2_base}")
        if lr:
            j5_pose[4] = -3.2
            j5_pose[5] = -71
        else:
            j5_pose[4] = -3.2
            j5_pose[5] = 71
    elif p_pixel[1] <= g_p.sight_middle_h and (p_pixel[0] < g_p.sight_middle_w[0] or p_pixel[0] > g_p.sight_middle_w[1]):
```

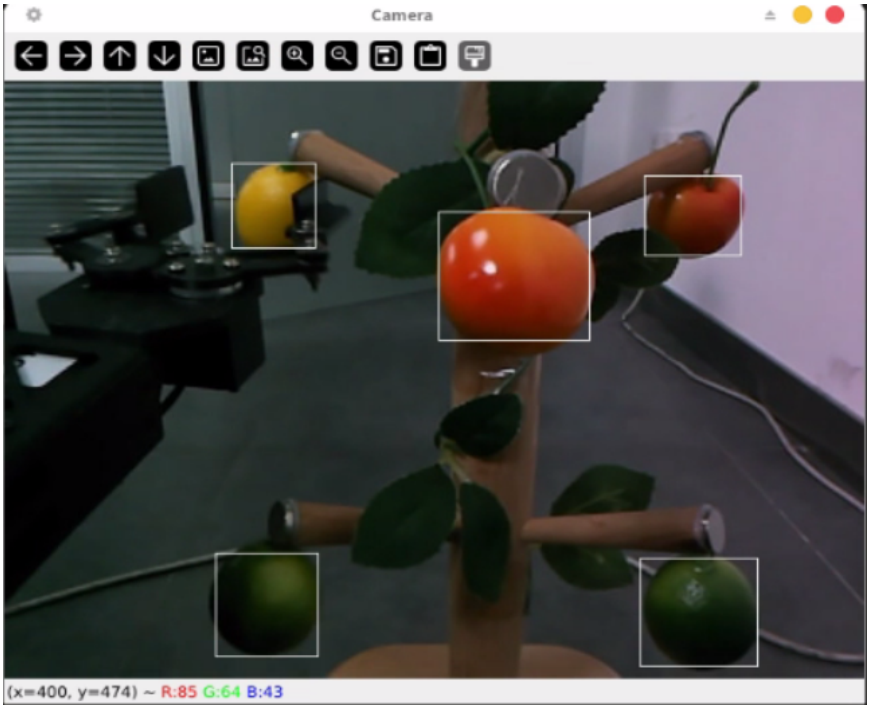
```
        x_obj_2_base = g_p.x_obj_to_cam_far + err_x
        print(f"上层两侧果到相机CCD距离校正: {g_p.x_obj_to_cam_far} + {err_x} =
{x_obj_2_base}")
        if lr:
            j5_pose[4] = -3.5
            j5_pose[5] = -28
        else:
            j5_pose[4] = -3.5
            j5_pose[5] = 28
    elif p_pixel[1] > g_p.sight_middle_h and (p_pixel[0] <
g_p.sight_middle_w[0] or p_pixel[0] > g_p.sight_middle_w[1]):
        x_obj_2_base = g_p.x_obj_to_cam_middle + err_x
        print(f"下层两侧果到相机CCD距离校正: {g_p.x_obj_to_cam_middle} + {err_x}
= {x_obj_2_base}")
        if lr:
            j5_pose[4] = 7.5
            j5_pose[5] = -55
        else:
            j5_pose[4] = 7.5
            j5_pose[5] = 55
```

同时，上段代码中 j5_pose[4: 5] 对应的抓取 pitch 和 yaw，也需手动调节，可以先进入机械臂调试，将车放置到特定抓取位置时，使用读取角度功能，掰动机械臂使夹爪够到对应位置水果，然后使用读取末端位姿功能，读取在该位置上夹爪够到对应位置水果时 J5 的末端位姿，关注此时 j5 末端在车体坐标系中的 P 和 Y，写入对应的位置上

```
roslaunch mission_manage grab_test.launch arm_dir:=left
```



点击图像中检测出来的框，代码会将边界框中心传入 J5 末端位姿解算，计算 J5 末端在车体坐标系中的位姿，然后解算 J5 末端在对应机械臂坐标系中的位姿，进行运动学反解获得 1-5 号舵机的角度，执行抓取预设动作组，详细过程可参考终端的输出



根据抓取情况，调节 相机/机械臂 的外参、距离参数，以及 j5_pose[4: 5]，直至实现抓取

Part 4 比赛任务全程联调

按任务流程，比赛分为：

获取裁判指令，大模型处理输出，解析，转为数字代号
出发前往采摘区
左/右臂采摘一号树上的水果
前往放置区对应颜色的放置框，放下水果
回到一号树，继续采摘，如没有可采摘水果，则前往二号树
以此类推，重复上述采摘/放置过程
直至采摘结束或时间即将截止，前往停车

全程联调代码位于
/home/orangepi/mission_ws/src/mission/mission_manage/scripts/mission_manage_node.py

对于大模型，在 Part 2 -> SubPart 3 中已经说明，在全程联调中使用以下代码发布模型输入，需修改下方代码中的附加提示词，将其改为之前调试后固化下来的提示词，在此不提供默认提示词（调试时可以注释掉大模型推理相关功能，直接使用手动指定的结果，同时也可注释掉 launch 文件中大模型的开启，以加快调试速度，比赛时再注释掉调试部分，打开大模型推理的相关功能）

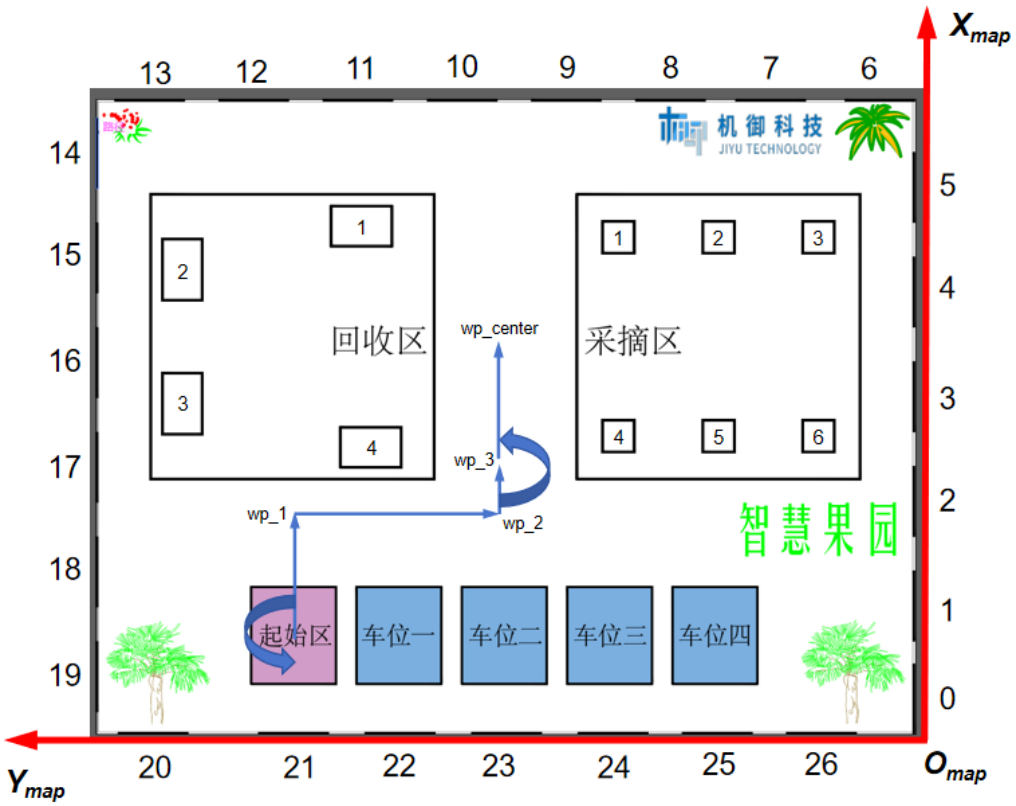
```
#
################################################################################
####
# 输入任务指令 等待deepseek输出
# ##############################
ds_input = String()
input_temp = input("输入任务指令\n>>>")
ds_input.data = "在此填入自定义附加提示词" \
                + input_temp + "。"
ds_input_pub.publish(ds_input)
```

```
9
10  t0 = time.time()
11  rospy.loginfo("任务开始\n")
12
13  # 等待deepseek处理得到的结果
14  ds_output: String = rospy.wait_for_message('/deepseek/output_text', String,
    timeout=None)
15  #
    ##############################################################################
    ####
16
17
18  # #
    ##############################################################################
    ####
19  # # 手动指定结果 仅供调试使用
20  # # ##############################
21  # t0 = time.time()
22  # rospy.loginfo("任务开始\n")
23  # ds_output = String()
24  # ds_output.data = "[[熟柠檬, 绿色], [坏柠檬, 蓝色], [坏樱桃, 红色], [熟樱桃, 黄
    色], [2]]"
25  # #
    ##############################################################################
    ####
```

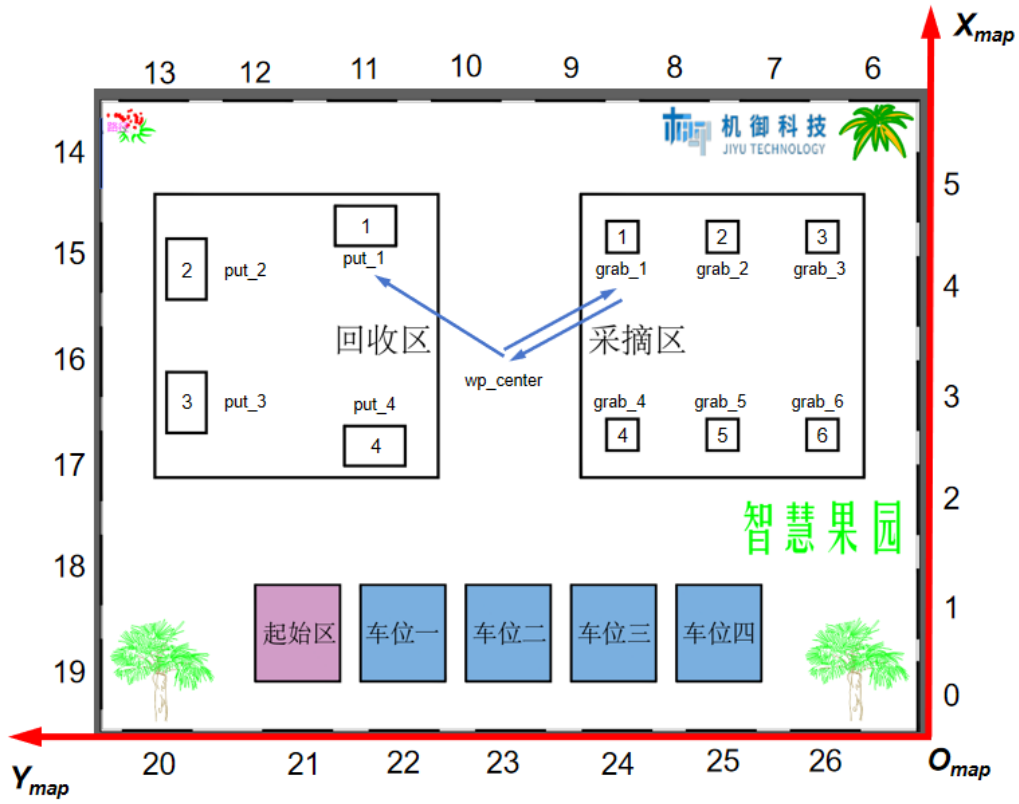
然后采集对应导航点的位姿以及相应的导航到位参数等，参数位于 /home/orangepi/mission_ws/src/mission/mission_manage/params/ mission_params.yaml，类似如下，每个航点列表包含航点 x/y/yaw 的位姿信息，以及用于定位的目标二维码 id（-1则为不指定 id），到位条件（包含 距离到位 exit_l、角度到位 exit_a 以及连续到位次数 cnt）

```
1  go_to_grab:  # 前往采摘的途经点
2      wp_num: 3
3      #     [x,     y,     yaw, id, exit_l, exit_a, cnt]  [m, m, °, id, m, °,
   cnt]
4      wp_1: [0.550, 2.520, 180, 21, 0.1,    5,      1]
5      wp_2: [0.450, 1.750, 180, 23, 0.1,    5,      1]
6      wp_3: [1.120, 1.750, 180, 23, 0.1,    5,      1]
```

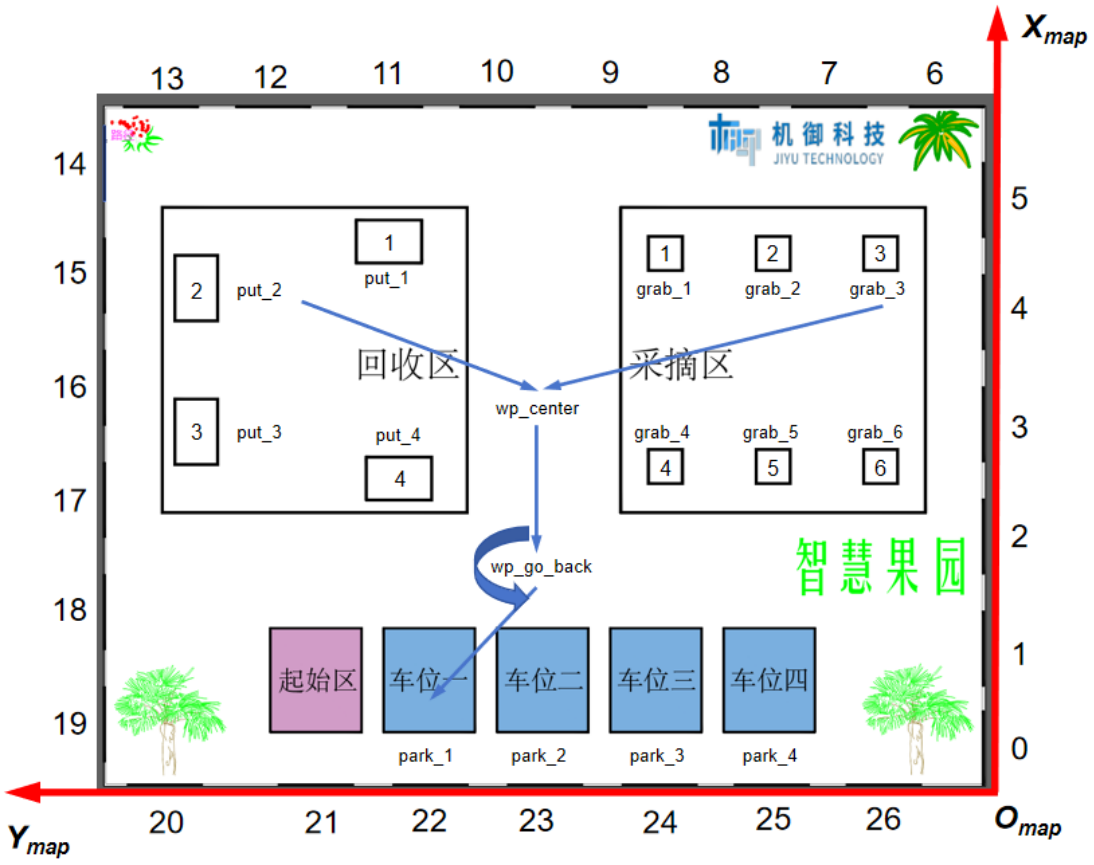
针对从起始区出发，在实践中发现，车体在起始区面朝 x 轴正方向时，若使用 6-13 号二维码作为定位源，则定位稳定性较低，故测完放置区顺序后调转 180°，使用 y 轴上的二维码，倒退离开起始区，倒退直到中场，再调转 180°，前往中转点



然后从中转点前往采摘区中的果树前，检测是否有可以采摘的果实
如果有，则采摘下，先返回中转点，然后前往回收区
往返采摘放置，当一棵树上检测不到可采摘水果时，前往下一棵树



直至时间将要用尽，返回泊车，返回时，先前往中转点，然后倒退至返回点，在返回点转向 180° 掉头，最后前往指定停车位




```
roslaunch mission_manage mission.launch
```